

A high-level programming language for generative protein design

Brian Hie^{1,2,*} Salvatore Candido^{1,*} Zeming Lin^{1,3} Ori Kabeli¹
Roshan Rao¹ Nikita Smetanin¹ Tom Sercu¹ Alexander Rives^{1,4,†}

Abstract

Combining a basic set of building blocks into more complex forms is a universal design principle. Most protein designs have proceeded from a manual bottom-up approach using parts created by nature, but top-down design of proteins is fundamentally hard due to biological complexity. We demonstrate how the modularity and programmability long sought for protein design can be realized through generative artificial intelligence. Advanced protein language models demonstrate emergent learning of atomic resolution structure and protein design principles. We leverage these developments to enable the programmable design of de novo protein sequences and structures of high complexity. First, we describe a high-level programming language based on modular building blocks that allows a designer to easily compose a set of desired properties. We then develop an energy-based generative model, built on atomic resolution structure prediction with a language model, that realizes all-atom structure designs that have the programmed properties. Designing a diverse set of specifications, including constraints on atomic coordinates, secondary structure, symmetry, and multimerization, demonstrates the generality and controllability of the approach. Enumerating constraints at increasing levels of hierarchical complexity shows that the approach can access a combinatorially large design space.

Introduction

Protein design would benefit from the regularity, simplicity, and programmability provided by a basic set of abstractions (1–4) like those used in the engineering of buildings, ma-

chines, circuits, and computer software. But unlike these artificial creations, proteins cannot be decomposed into easily recombinable parts because the local structure of the sequence is entangled in its global context (5, 6). Classical de novo protein design has attempted to determine a fundamental set of structural building blocks, which could then be assembled into higher-order structures (7–11). Likewise, traditional protein engineering often recombines segments or domains of natural protein sequences into hybrid chimeras (12–14). However, existing approaches have not been able to achieve the high combinatorial complexity that is necessary for true programmability.

We show modern generative models realize these classical goals of modularity and programmability at a new level of combinatorial complexity. Our idea is to place the modularity and programmability at a higher level of abstraction, where a generative model bridges the gap between human intuition and the production of specific sequences and structures. In this setting, the protein designer needs only to recombine high-level directives, while the task of obtaining a protein that fulfills those directives is placed on the generative model.

We propose a programming language for generative protein design, which allows a designer to specify intuitive, modular, and hierarchical programs. We show that high-level programs can be translated into low-level sequences and structures by a generative model. Our approach leverages advances in protein language models, which learn structural information (15, 16) and the design principles of proteins (see accompanying paper by Verkuil et al.).

In this study, our specific implementation is based on an energy-based generative model. First, a protein designer specifies a high-level program consisting of a set of hierarchically organized constraints (Figure 1A). Then, this program compiles to an energy function that evaluates compatibility with the constraints, which can be arbitrary and non-differentiable (Figure 1B). We apply constraints on structure by incorporating atomic-level structure predictions, enabled by a language model, into the energy function. This approach enables the generation of a wide set of complex designs (Figure 1C).

The use of a high-level language allows the protein designer

生成蛋白质设计的高级编程语言

Brian Hie^{1,2,*} Salvatore Candido^{1,*} Zeming Lin^{1,3} Ori Kabeli¹
Roshan Rao¹ Nikita Smetanin¹ Tom Sercu¹ Alexander Rives^{1,4,†}

摘要

将一组基本构建块组合成更复杂的形式是一个普遍的设计原则。大多数蛋白质设计是从手动自下而上的方法开始的，使用自然界创造的部分，但由于生物复杂性，蛋白质的自上而下设计在根本上是困难的。我们展示了通过生成性人工智能实现长期以来寻求的蛋白质设计的模块化和可编程性。先进的蛋白质语言模型展示了对原子分辨率结构和蛋白质设计原则的突现学习。我们利用这些发展，以支持对新生蛋白质序列和高度复杂结构的可编程设计。首先，我们描述了一种基于模块化构建块的高级编程语言，允许设计师轻松组合所需属性的集合。然后，我们开发了一个基于能量的生成模型，它建立在与语言模型结合的原子分辨率结构预测上，实现了具有编程属性的全原子结构设计。设计一组多样的规范，包括对原子坐标、二级结构、对称性和多聚化的约束，展示了该方法的普遍性和可控性。在逐步增加的层次复杂性中列举约束显示，该方法可以访问组合性极大的设计空间。

器、电路和计算机软件中使用的方式相似。但与这些人工制品不同，蛋白质不能被分解为易于重组的部分，因为序列的局部结构与其全局上下文纠缠在一起（5, 6）。经典新生蛋白设计试图确定一组基础结构构件，这些构件可以被组装成更高阶的结构（7–11）。同样，传统的蛋白质工程通常将自然蛋白质序列的片段或领域重新组合成混合嵌合体（12–14）。然而，现有的方法尚未能够实现真正可编程所需的高组合复杂性。

我们展示了现代生成模型在组合复杂性的新层面上实现了模块化和可编程性的这些经典目标。我们的想法是将模块化和可编程性置于更高的抽象层次，在该层次上，生成模型弥合了人类直觉与特定序列和结构的生成之间的鸿沟。在这种环境中，蛋白质设计师仅需重新组合高级指令，而获得满足这些指令的蛋白质的任务则由生成模型承担。

我们提出了一种用于生成蛋白质设计的编程语言，允许设计师指定直观的、模块化的和层次化的程序。我们展示了高级程序可由生成模型转换为低级序列和结构。我们的方法利用了蛋白质语言模型的进展，这些模型学习结构信息 (15, 16) 和蛋白质的设计原则（参见 Verkuil 等人的附随论文）。

在本研究中，我们的具体实现基于一种基于能量的生成模型。首先，蛋白质设计师指定一个由一组分层组织的约束组成的高级程序（图1A）。然后，该程序编译为一个评估与约束兼容性的能量函数，该函数可以是任意的且不可微分（图1B）。我们通过将语言模型启用的原子级结构预测纳入能量函数，对结构施加约束。这种方法使得生成广泛的复杂设计成为可能（图1C）。

引言

蛋白质设计将受益于通过一组基本抽象提供的规律性、简洁性和可编程性（1–4），这与在建筑、机-

*平等贡献¹Meta 基础人工智能研究 蛋白质团队 (FAIR)。² 斯坦福大学。作为 Meta AI 的访问研究员进行的工作。³ 纽约大学。作为 Meta AI 的访问研究员进行的工作。⁴ 纽约大学。

†通讯至 <arives@meta.com>。

预印本。版权所有 2022，作者。

使用高级语言使得蛋白质设计者能够

^{*}Equal contribution¹Meta Fundamental AI Research Protein Team (FAIR).²Stanford University. Work performed as a visiting researcher at Meta AI.³New York University. Work performed as a visiting researcher at Meta AI.⁴New York University.
[†]Correspondence to <arives@meta.com>.

A high-level programming language for generative protein design

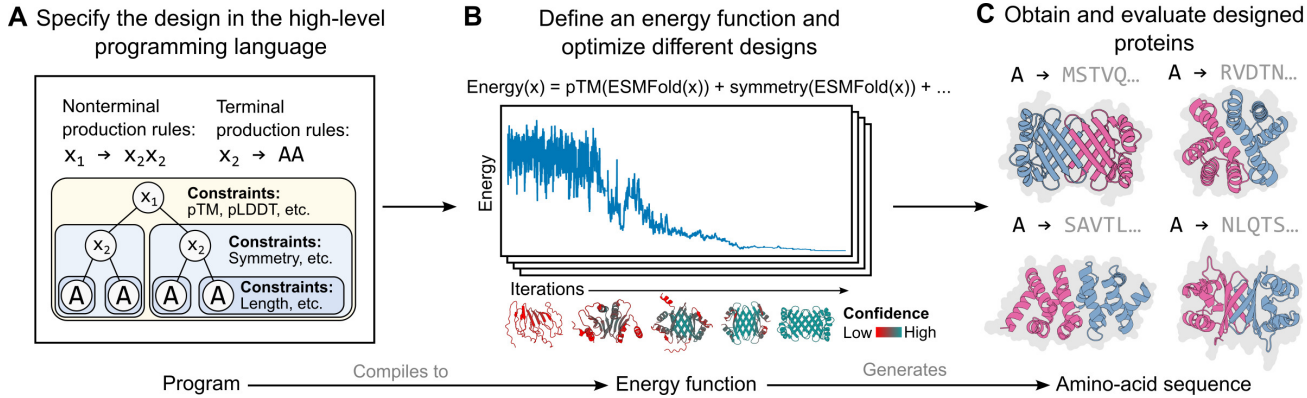


Figure 1. Overview of the high-level programming language and the optimization algorithm. (A) We propose a high-level programming language in which each program consists of (1) a syntax tree (corresponding to a set of nonterminal and terminal production rules) that enables modular and hierarchical organization of protein subunits and (2) a set of constraint functions that can be defined at each node of the syntax tree, where a given constraint is applied to the entire subtree rooted at the corresponding node. (B) This program is then compiled to a single energy function, which in our study is a simple linear combination of the specified constraint functions. The energy function is used to guide an optimization procedure based on simulated annealing, of which a key component is the use of an accurate and efficient structure predictor to evaluate the energy function at each step of the optimization. The same energy function can guide multiple optimization trajectories. (C) Each of these trajectories produces a protein sequence design and an associated predicted structure. These sequences and predicted structures can then be evaluated downstream using in silico and experimental metrics.

to systematically reason about the design space and specify very general, modular, and composable programs. To demonstrate this, we generate proteins that realize a variety of constraints that include secondary structure, symmetry, multimerization, and atomic-level coordination in the predicted structures. We apply these constraints in complex, hierarchical settings, where we can enumerate a space of highly idealized forms that have low similarity to natural structures. As de novo design progresses to more complex proteins and protein assemblies, high-level abstractions such as the programming language described in this study should facilitate the systematic exploration and design of complex artificial proteins.

A generative programming language for protein design

We introduce a high-level programming language for generative protein design. This language first requires a syntax tree (Figure 1A) consisting of terminal symbols (i.e., the leaves of the tree) that each corresponds to a unique protein sequence (which is potentially repeated within the protein) and nonterminal symbols (i.e., the internal nodes of the tree) that enable hierarchical organization. Second, the language requires a set of constraints: at each node in the tree, a protein designer can specify any number of constraints, which are applied to the entire subtree. The syntax tree and its constraints fully specify a program in our high-level language. We provide a more extended description of this language in

the Methods section.

Each program is compiled into an energy function that specifies a generative model for that program in the form of a distribution

$$p_{\theta}(\mathbf{x}) = \frac{\exp(-E_{\theta}(\mathbf{x}))}{Z(\theta)}$$

over protein sequences $\mathbf{x}$ conforming to the program. The constraints are encoded as weighted terms that are additively combined into the total energy. Since the partition function Z , which is a function of the parameters θ , is intractable, low temperature samples can be taken with MCMC and simulated annealing (Figure 1B). The generative capacity of this approach is built on recent developments in deep learning for protein biology. Specifically, each step in the optimization loop has access to a fast and accurate atomic-level structure prediction enabled by the ESM-2 protein language model.

Given a single program, the generative model can create potentially diverse designs that fulfill the user-specified constraints (Figure 1C). These constraints can be arbitrary and nondifferentiable, and can span multiple scales of biological complexity, from atomic-level coordinates to abstract plans of the protein including the overall topology and symmetry. This approach allows the model to propose diverse solutions where many potential designs may satisfy the program. By leveraging an expressive model of structure in a generative capacity, the resulting designs respect the various constraints individually and are also globally coherent.

一种用于生成蛋白质设计的高级编程语言

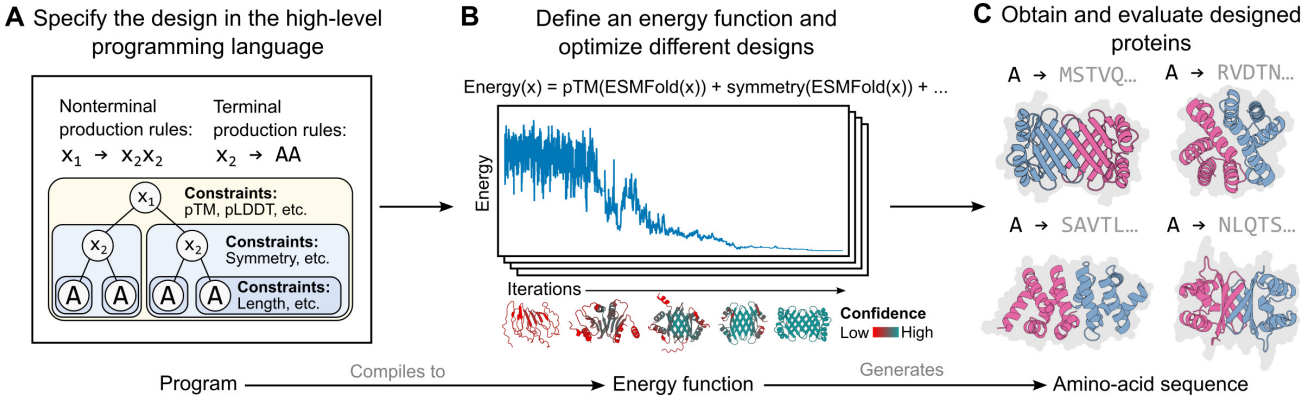


图1. 高级编程语言和优化算法的概述。 (A) 我们提出了一种高级编程语言，其中每个程序由 (1) 一个语法树（对应于一组非终结符和终结符的产生式规则）组成，允许蛋白质子单元的模块化和层次化组织，以及 (2) 一组可在语法树的每个节点上定义的约束函数，其中给定的约束适用于以相应节点为根的整个子树。(B) 然后将该程序编译成一个单一的能量函数，在我们的研究中，这是指定约束函数的简单线性组合。能量函数用于指导基于模拟退火的优化过程，其中一个关键组成部分是使用准确且高效的结构预测器来评估优化每一步的能量函数。相同的能量函数可以引导多个优化轨迹。(C) 这些轨迹中的每一个都会生成蛋白质序列设计相关的预测结构。随后可以使用计算机仿真和实验指标对这些序列和预测结构进行后续评估。

系统性地推理关于设计空间并指定非常通用、模块化和可组合的程序。为了证明这一点，我们生成实现多种约束的蛋白质，这些约束包括二级结构、对称、多聚化和预测结构中的原子级协调。我们在复杂的层次设置中应用这些约束，在这些设置中，我们可以枚举出与自然结构相似度较低的高度理想化形式的空间。随着从头设计向更复杂的蛋白质和蛋白质组装的进展，如本研究中所述的编程语言等高级抽象应促进复杂人工蛋白质的系统探索和设计。

用于蛋白质设计的生成编程语言

我们引入了一种用于生成蛋白质设计的高级编程语言。该语言首先需要语法树（图1A），该语法树由终端符号（即树的叶子）组成，每个终端符号对应一个独特的蛋白质序列（该序列可能在蛋白质中重复）和非终端符号（即树的内部节点），它们使得层次化组织成为可能。其次，该语言需要一组约束：在树的每个节点上，蛋白质设计者可以指定任意数量的约束，这些约束应用于整个子树。语法树及其约束完全定义了我们高级程序中的一个程序。我们在

方法部分。

每个程序被编译成一个能量函数，该能量函数以分布的形式指定该程序的生成模型。

$$p_{\theta}(\mathbf{x}) = \frac{\exp(-E_{\theta}(\mathbf{x}))}{Z(\theta)}$$

关于蛋白质序列 $\mathbf{x}$ 以符合该程序。约束被编码为加权项，这些项以加法方式组合成总能量。由于分区函数 Z ，它是参数的一个函数 θ ，在计算上无法处理，低温样本可以通过马尔科夫链蒙特卡洛（MCMC）和模拟退火技术来获取（图1B）。这种方法的生成能力建立于蛋白质生物学深度学习的最新发展之上。具体而言，优化循环中的每一步都可以访问由ESM-2蛋白质语言模型提供的快速且准确的原子级结构预测。

给定一个单一程序，生成模型可以创建满足用户指定约束的潜在多样化设计（图1C）。这些约束可以是任意的且不可微分的，并且可以跨越多个生物复杂性尺度，从原子级坐标到包括整体拓扑和对称性的蛋白质抽象方案。这种方法允许模型提出多样的解决方案，其中许多潜在设计可能满足该程序。通过利用具有生成能力的表达性结构模型，得到的设计各自遵循不同的约束，并且在整体上也是一致的。

A high-level programming language for generative protein design

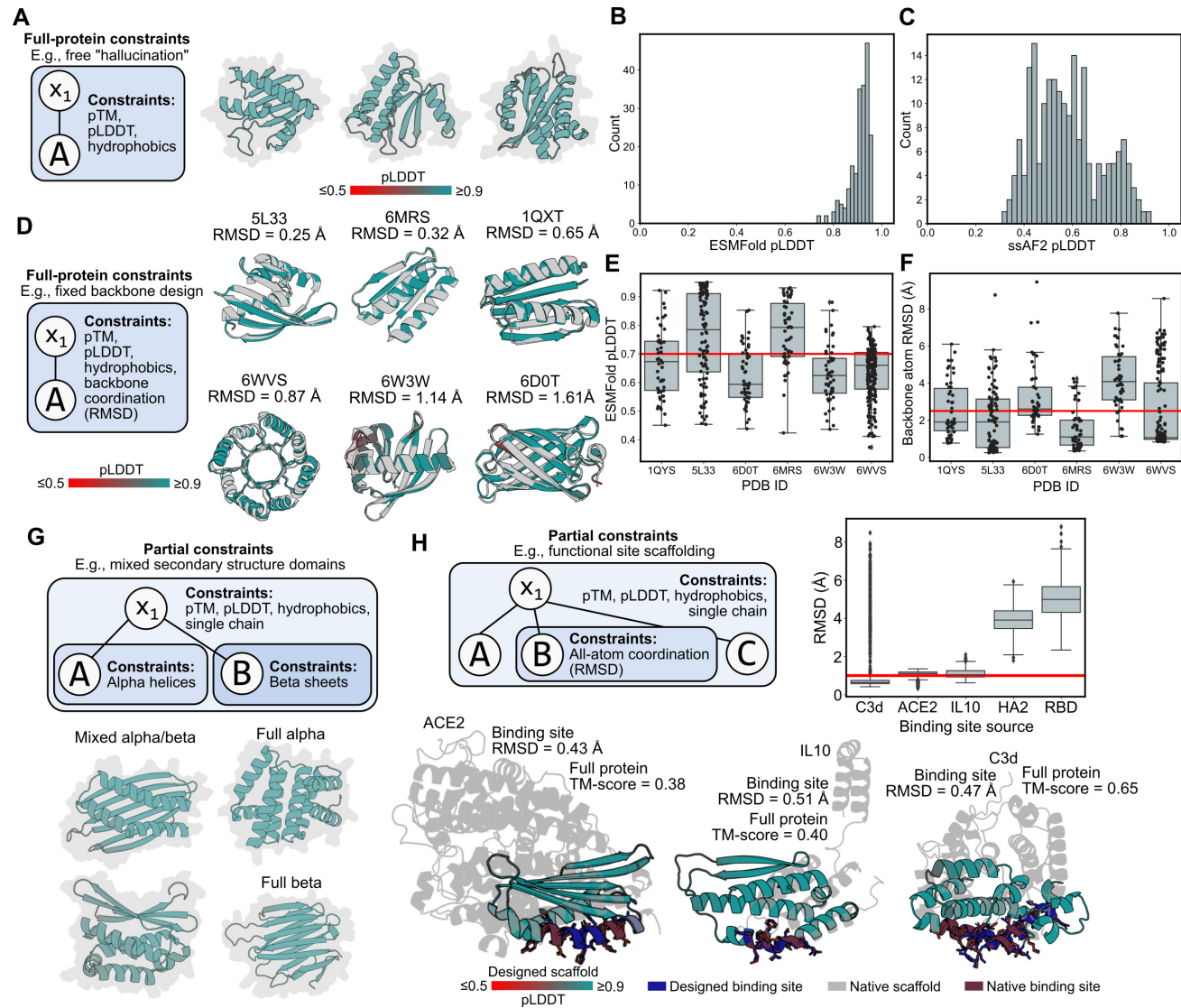


Figure 2. Programming full-protein or partial constraints. (A) A graphical representation of a program for protein “free hallucination” (left) along with three example designed structures (right). (B) The distribution of ESMFold pLDDT values over 200 free-hallucinated structures. Of these, 100% have good confidence (ESMFold pLDDT > 0.7). (C) The distribution of single-sequence AlphaFold2 (ssAF2) over the same 200 structures; note that ssAF2 was not used in the design procedure. Of these, 22% have good confidence (ssAF2 pLDDT > 0.7). (D) A graphical representation of a program for fixed backbone design (left) along with example designs for six de novo target backbones. The experimental backbone is colored gray; the designed backbone is colored by ESMFold pLDDT. (E) For each target backbone, the distribution of the ESMFold pLDDT values of the final designs from 50 or more fixed backbone design seeds is plotted as a boxplot (for all boxplots in this figure, the box extends from first to third quartile, black line indicates the median, and whiskers indicate 2.5 times the interquartile range) with each seed also plotted as a black circle. A horizontal red line indicates pLDDT = 0.7. (F) For each target backbone, the distribution of RMSD values between the target and design backbone atoms from 50 or more fixed backbone design seeds is plotted as a boxplot with each seed also plotted as a black circle. A horizontal red line indicates RMSD = 2.5 Å. (G) A graphical representation of a program for designing a protein with mixed secondary structure (top) along with example designs in which secondary structure was explicitly specified (bottom). (H) Top-left: A graphical representation of a program for functional site scaffolding. Top-right: For each scaffolded binding site, the distribution of RMSD between the native and designed binding site atoms (including side chains) from 2,000 seeds is plotted as a boxplot. A horizontal red line indicates RMSD = 2 Å. Bottom: Example designs that achieve sub-angstrom atomic coordination in the scaffolded binding site atoms, high model confidence in the associated scaffold, and low similarity (quantified by TM-score) to the natural protein.

用于生成蛋白质设计的高级编程语言

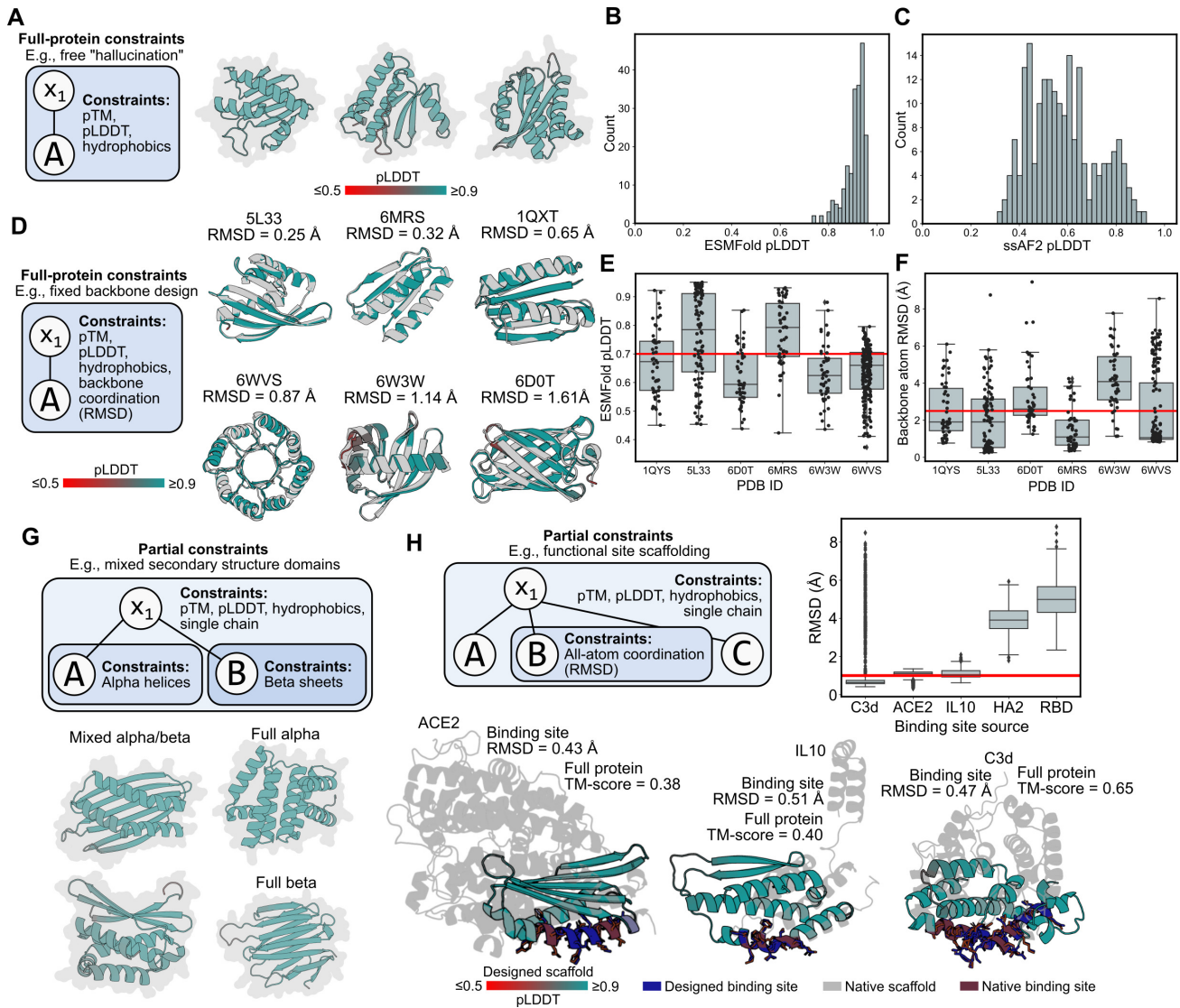


图 2. 编程全蛋白或部分约束。(A) 蛋白质“自由幻觉”的程序的图形表示(左)以及三个示例设计结构(右)。(B) 200个自由幻觉结构的ESMFold pLDDT值的分布。在这些结构中，100%的信心良好(ESMFold pLDDT > 0.7)。(C) 同样200个结构的单序列AlphaFold2 (ssAF2)的分布；请注意，ssAF2在设计过程中未被使用。在这些结构中，22%的信心良好(ssAF2 pLDDT > 0.7)。(D) 固定骨架设计程序的图形表示(左)以及六个de novo目标骨架的示例设计。实验骨架为灰色；设计骨架根据ESMFold pLDDT上色。(E) 对于每个目标骨架，从50个或更多固定骨架设计种子获得的最终设计的ESMFold pLDDT值的分布绘制为箱线图(对于本图中的所有箱线图，箱体从第一四分位数延伸到第三四分位数，黑线表示中位数，须表示2.5倍的四分位间距)，每个种子也绘制为一个黑色圆圈。水平红线表示pLDDT = 0.7。

(F) 对于每个目标骨架，来自50个或更多固定骨架设计种子的目标与设计骨架原子之间的RMSD值分布被绘制为箱线图，每个种子也以黑色圆圈表示。一条横向红线表示RMSD = 2.5 Å。(G) 设计一种具有混合二级结构的蛋白质的程序的图形表示(顶部)，以及显式指定二级结构的示例设计(底部)。(H) 左上：功能位点支架程序的图形表示。右上：对于每个支架结合位点，绘制了来自2000个种子的天然与设计结合位点原子(包括侧链)之间的RMSD分布作为箱线图。一条横向红线表示RMSD = 2 Å。底部：在支架结合位点原子中实现亚埃级原子协调的示例设计，在相关支架中具有高模型置信度，并且与天然蛋白质的相似性低(通过TM-score量化)。

A high-level programming language for generative protein design

Results

Full-protein constraints

We first demonstrate that our approach can design proteins where the constraints are simply applied to the entire sequence and structure without any hierarchical organization. An especially valuable constraint, which we apply generally across all our design efforts, steers the optimization toward predicted structures with higher model confidence, i.e., high pTM and mean pLDDT. For proteins that we desire to have soluble, monomeric expression, we also steer the optimization to minimize hydrophobic residues that are solvent-exposed (Methods). Using only these constraints on structural confidence and hydrophobic residue placement, our model is able to generate or “freely hallucinate” (17) high-confidence structures (Figures 2A and 2B); across 200 seeds, all optimization loops produced predicted structures with an ESMFold mean pLDDT greater than 0.7 (Figure 2B). Of these, a large portion (44, or 22%) also had high predicted confidence (pLDDT > 0.7) by single-sequence AlphaFold2 (18) (Figure 2C), a separate structure prediction model that was not used in our optimization procedure.

Our objective function also enables other full-protein constraints, such as specifying the positions of the backbone atoms while allowing the algorithm to design the corresponding sequence, a design task referred to as fixed backbone design (19). To achieve this, we can add a term to the energy function that minimizes the root-mean-square deviation (RMSD) between the corresponding designed and target backbone atoms (Methods). Our simulated annealing procedure successfully produces high-confidence designs with low RMSD (< 1.6 Å) across diverse de novo backbones (Figures 2D and 2E), and can do so reproducibly over different optimization runs (Figure 2F).

Partial constraints

We next sought to increase the complexity of our designable space by varying the constraints enforced on different parts of a protein. For example, a simple mixed-constraint setting is to specify a two-domain protein with different combinations of secondary structure composition (Figures 2G and S1A–S1C). In our programs, we can represent this setting by a syntax tree containing two or more subtrees, where different constraints are only applied within the discrete subtrees.

A more complex mixed-constraint setting is to design functional proteins by constraining one region of the protein design to have the same all-atom positions (including protein side chains) as a functional site from nature, while allowing the design procedure to freely generate the remainder of the protein; this design setting is sometimes referred to as functional site “scaffolding” (20). Importantly, in contrast to

fixed backbone design, in which constraints are only placed on backbone atomic coordinates, functional site scaffolding requires constraints on side-chain atoms as well, since these are critical to achieving function. Because our optimization procedure produces an all-atom structure prediction at each step of the optimization, we can readily incorporate this constraint as part of the energy function by minimizing the all-atom RMSD between the natural and designed atomic coordinates of the functional site (Methods).

Across functional sites involving sequence-contiguous or -discontiguous residues from a variety of natural proteins, our algorithm is able to produce designs that scaffold the site with sub-angstrom RMSD between the experimental and predicted structure in three out of five functional sites attempted (Figures 2H and S1D). Moreover, the algorithm produces designed scaffolds that depart from the native protein (Figure 2H). The ability to move natural functional sites onto designed backbones has many practical applications, including the design of functional proteins that are smaller or stabler than their natural counterparts.

Symmetric and multimeric group constraints

Beyond proteins containing partial constraints, we next increase the complexity of our protein designs by generating structures that contain constraints over multiple subunits. A foundational design task for the generation of idealized, de novo proteins is to constrain structural symmetry (7, 22). To generate symmetric proteins, we first enforce the notion of a repeated unit that is repeated K times when designing a K -fold symmetry (where we can control the value of K). To guide the optimization toward symmetric structures, we add various constraints on the distances among the centroids of each repeated unit as part of the energy function (Methods). In our high-level language, a symmetric protein would be encoded by repeating the same non-terminal symbol K times (corresponding to the repeated unit); the symmetry constraint is then placed at the level of the syntax tree containing these repeated non-terminals (Figure 3A).

Using these symmetric constraints, we show that we can program the level of symmetry within a protein design. When directed to design 3- to 8-fold symmetry, the generative model produces a diverse set of high-confidence structures (Figures 3B, S2A, and S2B), including folds that have common analogs in nature (including coiled-coils, beta propellers, beta sandwiches, beta barrels, and TIM barrels) as well as highly-idealized designs that are different from natural structures, including a pentagonal star-shaped protein (with a TM-score of 0.48 to the nearest PDB structure 3S38; row 1 and column 3 in Figure 3B) and a cube-shape protein (nearest-PDB TM-score of 0.51 to PDB 7DEG; row 2 and column 2 in Figure 3B). The highlighted symmetric proteins in Figure 3B have nearest-PDB TM-scores ranging from

用于生成蛋白质设计的高级编程语言

结果

全蛋白约束

我们首先证明我们的方法能够设计出在没有任何层次组织的情况下将约束简单应用于整个序列和结构的蛋白质。一个特别有价值的约束, 我们普遍应用于所有设计努力中, 它引导优化朝着具有更高模型置信度的预测结构进行, 即高 pTM 和平均 pLDDT。对于我们希望具有可溶的单体表达的蛋白质, 我们还引导优化以最小化暴露于溶剂的疏水残基 (方法)。仅使用这些关于结构信心和疏水残基位置的约束, 我们的模型能够生成或“自由幻觉” (17) 高置信度结构 (图 2A 和 2B); 在200个种子中, 所有优化循环都产生了平均 pLDDT 大于 0.7 的预测结构 (图2B)。其中, 大部分 (44, 或22%) 也具有高预测置信度 (pLDDT> 0.7) 通过单序列 AlphaFold2 (18) (图 2C), 这是一个在我们的优化过程中未曾使用的独立结构预测模型。

我们的目标函数还能够实现其他全蛋白质约束, 例如在允许算法设计相应序列的同时指定主链原子的位置信息, 这一设计任务称为固定主链设计 (19)。为了实现这一点, 我们可以在能量函数中添加一个项, 以最小化相应设计的主链原子与目标主链原子之间的均方根偏差 (RMSD) (方法)。我们的模拟退火程序成功地产生了低 RMSD 的高置信度设计 (< 1.6 Å) 针对多样的新设计主链 (图2D 和 2E), 并且可以在不同的优化运行中可靠地实现这一点 (图2F)。

部分约束

我们接下来寻求通过变化施加在蛋白质不同部分上的约束来增加我们可设计空间的复杂性。例如, 一个简单的混合约束设置是指定一个具有不同二级结构组成组合的双域蛋白 (图2G 和 S1A–S1C)。在我们的程序中, 我们可以通过包含两个或多个子树的语法树来表示这个设置, 其中不同的约束仅在离散子树内应用。

一种更复杂的混合约束设置是通过约束蛋白质设计的一个区域, 使其具有与来自自然的功能位点相同的全原子位置 (包括蛋白质侧链) 来设计功能性蛋白质, 同时允许设计过程自由生成其余部分的蛋白质; 这种设计设置有时被称为功能位点“支架” (20)。重要的是, 与

固定骨架设计不同, 其中仅对骨架原子坐标施加约束, 功能位点支架还要求对侧链原子施加约束, 因为这些原子对实现功能至关重要。由于我们的优化过程在优化的每一步产生全原子结构预测, 我们可以通过最小化自然与设计的功能位点原子坐标之间的全原子 RMSD, 将这一约束方便地纳入能量函数 (方法)。

在涉及来自多种天然蛋白质的序列连续或不连续残基的功能位点上, 我们的算法能够生成设计, 使得实验结构与预测结构之间的RMSD达到亚埃级精度, 在尝试的五个功能位点中有三个达到了这一标准 (图 2H 和 S1D)。此外, 该算法生成的设计支架偏离了天然蛋白质 (图2H)。将天然功能位点移至设计骨架的能力具有许多实际应用, 包括设计比其天然对应物更小或更稳定的功能性蛋白。

对称和多聚体组约束

除了包含部分约束的蛋白质外, 我们接下来通过生成包含多个亚单位的约束的结构来增加我们蛋白质设计的复杂性。生成理想化的从头蛋白质的一个基础设计任务是约束结构对称性 (7, 22)。为了生成对称蛋白, 我们首先强加一个重复单元的概念, 该单元在设计 K 折叠对称时重复 K 次 (我们可以控制 K 的值)。为了引导优化朝向对称结构, 我们在能量函数中添加了各种约束, 约束涉及每个重复单元的质心之间的距离 (方法)。在我们的高级语言中, 一个对称蛋白将通过重复相同的非终结符号 K 次进行编码 (对应于重复单元); 对称约束然后被置于包含这些重复非终结符号的语法树的层次中 (图3A)。

利用这些对称约束, 我们展示了能够对蛋白质设计中的对称性水平进行编程。当指导设计3到8重对称时, 生成模型产生了一组多样且高置信度的结构 (图3B, S2A 和 S2B), 包括在自然界中具有常见类似物的折叠 (包括卷曲圈、β推进器、β三明治、β桶和TIM桶), 以及与自然结构不同的高度理想化设计, 包括一种五角星形蛋白 (与最近PDB结构3S38的TM-score为 0.48; 图3B中的第1行和第3列) 和一种立方形蛋白 (与PDB 7DEG的最近-PDB TM-score为0.51; 图 3B中的第2行和第2列)。图3B 中突出显示的对称蛋白的最近-PDB TM分数范围从

A high-level programming language for generative protein design

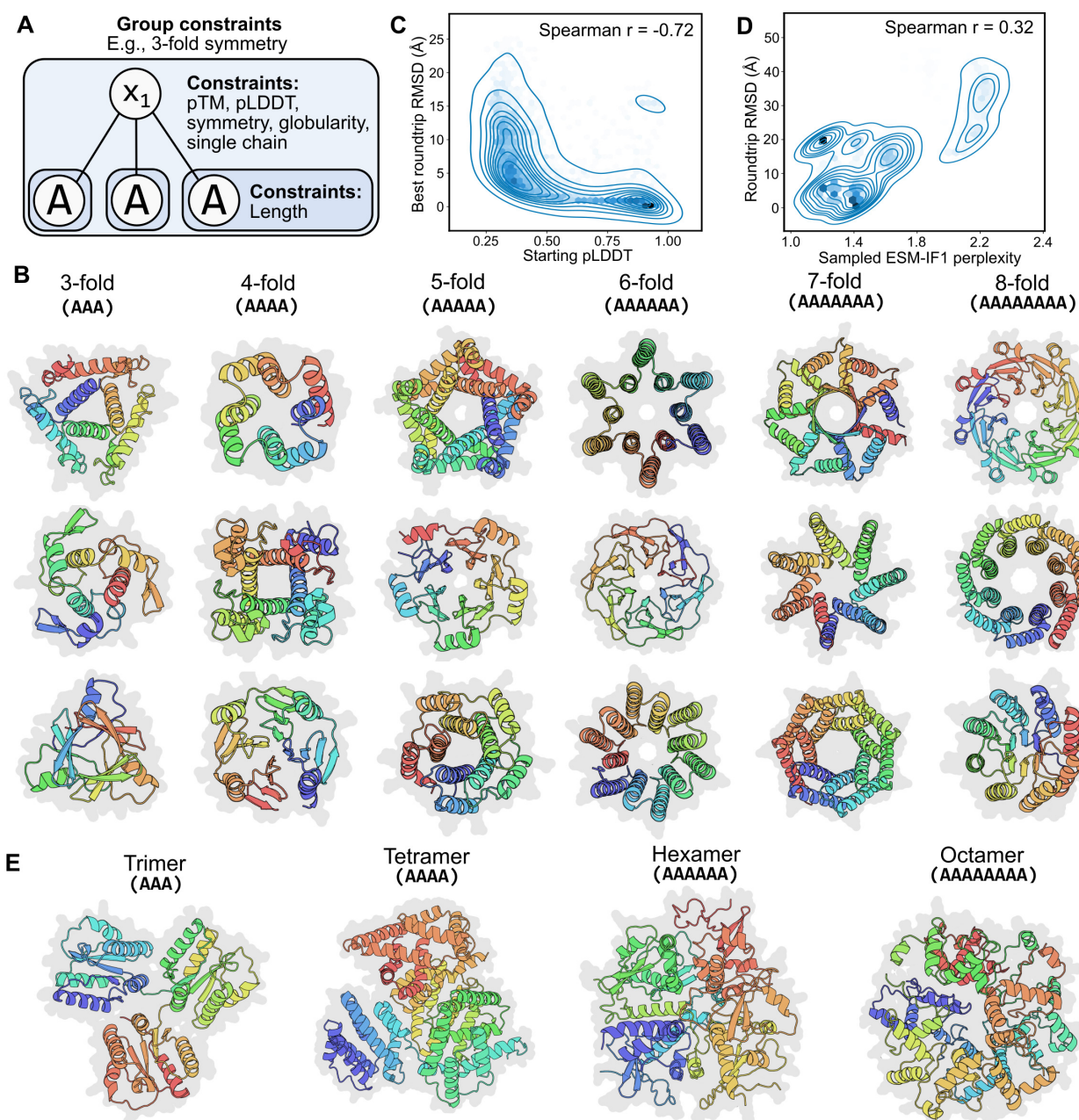


Figure 3. Programming symmetry and homo-oligomerization. (A) A graphical representation of a program for designing a single protein chain with 3-fold symmetry based on a repeated subsequence. (B) Example designs varying fold symmetry from 3- to 8-fold. (C) 1000 randomly sampled symmetric protein designs were “roundtripped” by sampling ten sequences via ESM-IF1 inverse folding (21) of their backbones followed by ESMFold structure prediction. The ESMFold pLDDT of the starting backbone is indicated on the horizontal axis. The lowest of the 10 RMSDs comparing the starting and roundtripped backbone atoms is indicated on the vertical axis. Blue lines indicate density contours and hexagonal bins are darker with greater density. We observed that a more confident design is associated with roundtrip success. (D) 1,000 randomly sampled inverse folding samples are plotted according to their ESM-IF1 perplexity on the horizontal axis and their roundtrip RMSD on the vertical axis. We observed that a lower perplexity sequence is associated with roundtrip success. (E) Example homo-oligomers with increasing numbers of individual protomers. The tetrameric, hexameric, and octameric oligomers depicted here form globular polyhedral shapes rather than the rotational symmetry of designs in (B).

用于生成蛋白质设计的高级编程语言

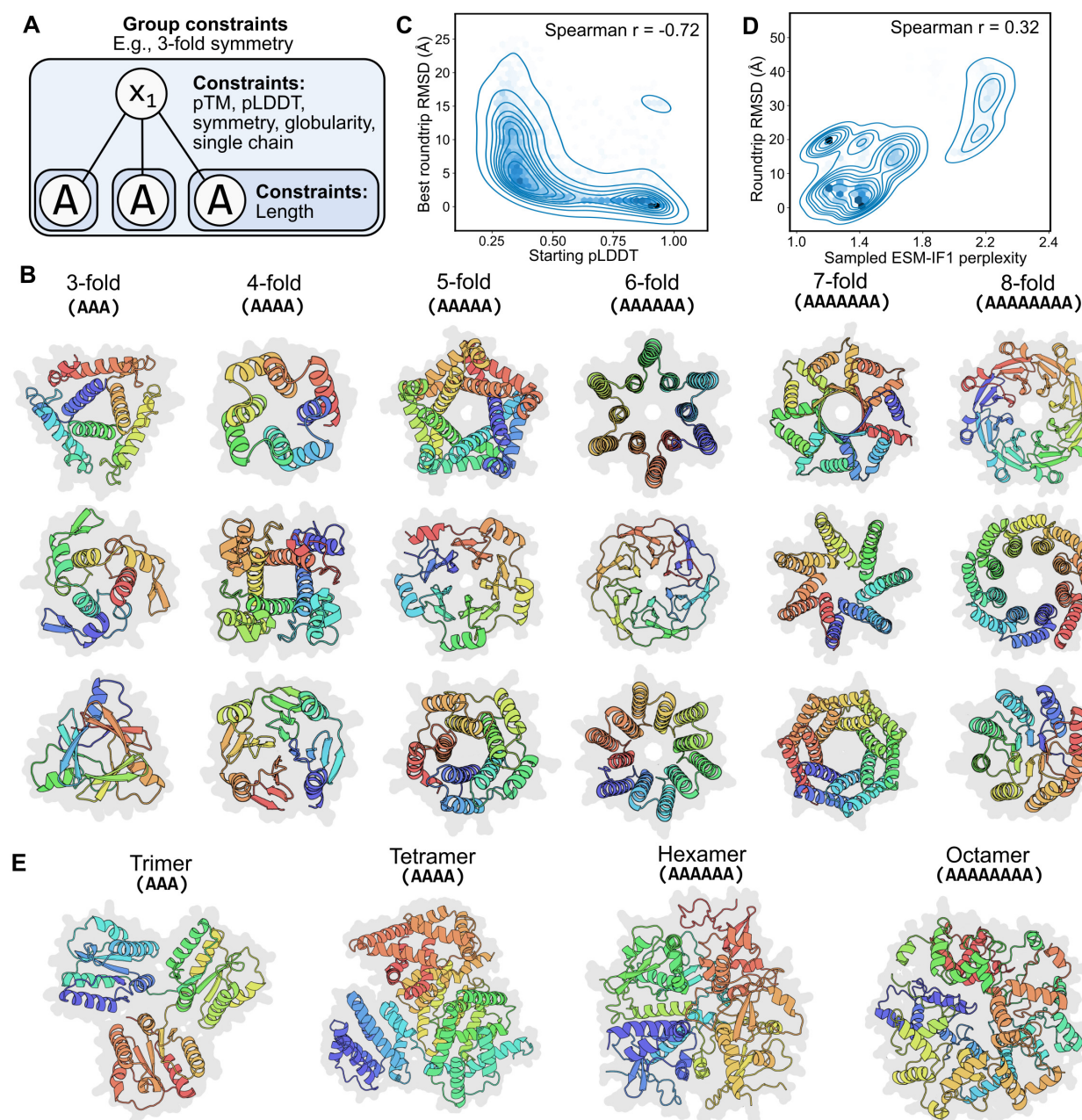


图 3.编程对称性和同源寡聚化。(A) 基于重复子序列的三重对称单个蛋白质链设计程序的图形表示。(B) 示例设计展示了从3倍到8倍的对称变化。(C) 随机抽取的1000个对称蛋白质设计通过ESM-IF1逆折叠(21)其骨架采样十个序列之后“回转”。ESMFold结构预测表明初始骨架的ESMFold pLDDT在横轴上表示。比较初始和回转骨架原子之间的10个RMSD中的最低值在纵轴上指示。蓝线表示密度轮廓，六边形箱的密度更高则为更深的颜色。我们观察到，设计的信心越高，与回转成功的关联越大。(D) 1000个随机抽取的逆折叠样本根据其在横轴上的ESM-IF1困惑度和在纵轴上的回转RMSD进行绘制。我们观察到，较低的困惑度序列与回转成功相关。(E) 示例同源寡聚物，显示个体原聚物数量的增加。此处描绘的四聚体、六聚体和八聚体寡聚物形成球状多面体形状，而不是(B)中设计的旋转对称性。

A high-level programming language for generative protein design

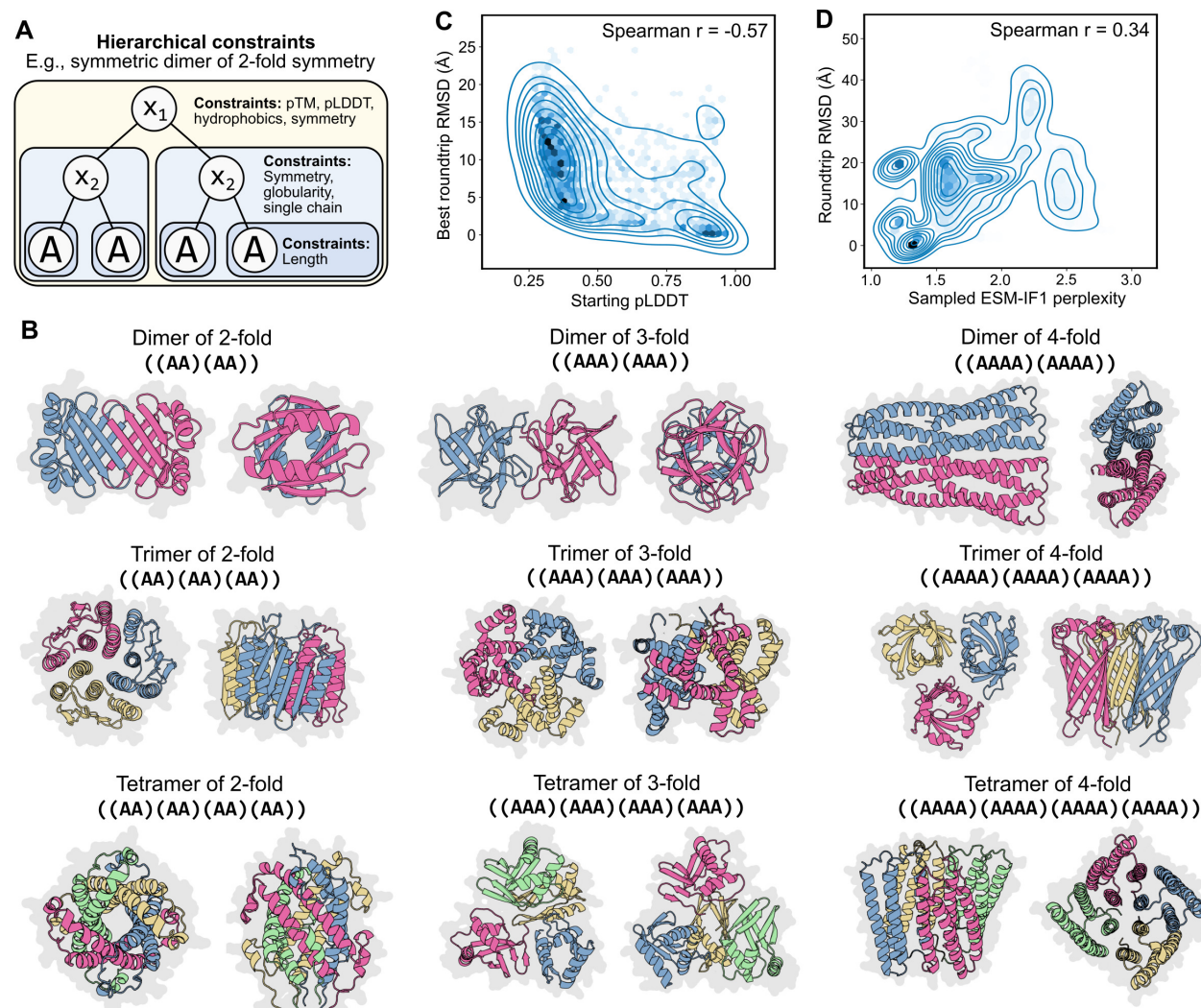


Figure 4. Programming two levels of symmetry. (A) A graphical representation of a program for designing two levels of symmetry in which a homo-oligomeric symmetric dimer represents the top level of symmetry and each unit within the dimer also has two-fold symmetry. (B) Example oligomers with two levels of symmetry, in which we procedurally enumerate across a grid in which we vary the top-level symmetry across the rows and the bottom-level symmetry across the columns. Discrete chains are indicated by different colors. (C) 1,000 randomly sampled two-level symmetric protein oligomer designs were “roundtripped” by sampling ten sequences via ESM-IF1 inverse folding (21) of their backbones followed by ESMFold structure prediction (Methods). The ESMFold pLDDT of the starting backbone is indicated on the horizontal axis. The lowest of the 10 RMSDs comparing the starting and roundtripped backbone atoms is indicated on the vertical axis. Blue lines indicate density contours and hexagonal bins are darker with greater density. We observed that a more confident design is associated with roundtrip success. (D) 1,000 randomly sampled inverse folding samples are plotted according to their ESM-IF1 perplexity on the horizontal axis and their roundtrip RMSD on the vertical axis. We observed that a lower perplexity sequence is associated with roundtrip success.

用于生成蛋白质设计的高级编程语言

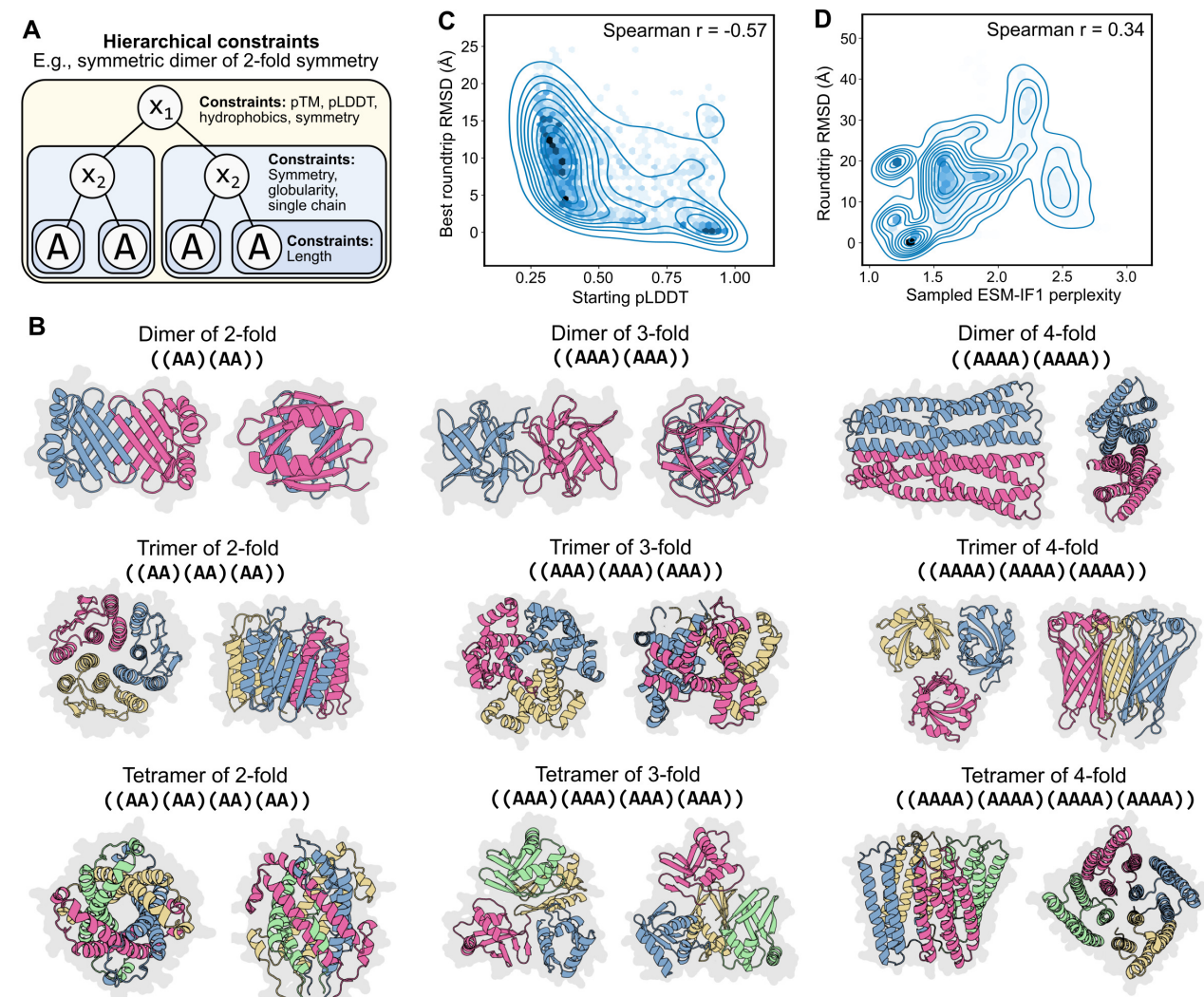


图4. 编程两级对称性。 (A) 设计两级对称性的程序的图形表示，其中同聚体对称二聚体代表对称性的最高层级，而二聚体中的每个单元也具有二重对称性。(B) 具有两级对称性的示例寡聚体，我们通过在一个网格上进行程序化枚举，其中我们在行中变化顶层对称性，在列中变化底层对称性。离散链通过不同颜色表示。

(C) 1,000 个随机采样的两级对称蛋白质寡聚体设计通过对其骨架进行 ESM-IF1 逆折叠的方式 (21) 进行“回程”，随后进行 ESMFold 结构预测 (方法)。起始骨架的 ESMFold pLDDT 在水平轴上表示。比较起始和回程骨架原子时 10 个 RMSD 中的最低值在垂直轴上表示。蓝色线表示密度轮廓，并且六边形箱的颜色随着密度增加而变暗。我们观察到，更有信心的设计与回程成功相关。(D) 1,000 个随机采样的逆折叠样本根据其 ESM-IF1 困惑度在水平轴上绘制，其回程 RMSD 在垂直轴上绘制。我们观察到，较低的困惑度序列与回程成功相关。

A high-level programming language for generative protein design

0.47 to 0.86, with a median TM-score of 0.64; TM-scores for all seeds are plotted in Figure [S2C](#).

To increase our confidence that these idealized structures correspond to valid and designable backbones, we observe that sampling sequences via inverse folding with the ESM-IF1 and ProteinMPNN models ([21](#), [23](#)) followed by structure prediction of the sequence samples can reproducibly recover the original backbone geometry (Methods), in many instances with sub-angstrom backbone-atom RMSD. To increase our confidence that a successful “roundtrip” through inverse folding indicates designable backbones, we observe that high confidence designs indicates better roundtrip success (Figures [3C](#) and [S2D](#)) and that the ability to sample a low perplexity sequence also indicates roundtrip success (Figure [3D](#)).

Beyond single-chain symmetries, we can also design multimeric proteins similarly. We enforce the notion of single or multiple chains in our programming language with a “single chain” constraint that dictates that all terminal elements in a subtree belong to the same chain (Methods); to design multimeric proteins, we need only remove this constraint. Example multimeric symmetric proteins involving 4- to 8-mers are provided in Figure [3E](#).

Hierarchical constraints

Formalizing our constraints into a syntax tree naturally enables the specification of hierarchical constraints, which enables more complex protein designs. As an initial demonstration, guided by our high-level language’s formalization, we design different levels of symmetry at two levels of hierarchy, where the lower level of symmetry is specified within a chain and the upper level of symmetry is specified among protomers in a homo-oligomer (Figure [4A](#)). We procedurally enumerate over examples that range from a dimer of units with 2-fold symmetry to a tetramer of units with 4-fold symmetry (Figures [4B](#), [S3A](#), and [S3B](#)). As in the single-chain symmetric design setting, we observe successful structure prediction roundtrips through inverse folding, and that both high-confidence predicted structures and low inverse folding perplexity indicate roundtrip success (Figures [4C](#) and [4D](#)). Many of the designs with two levels of symmetry have low overall similarity to structures in the PDB; for example, a dimer of 2-fold symmetry in which opposing beta sheets form a regular checkerboard pattern (nearest-PDB TM-score of 0.49 to PDB 3W38; row 1 and column 1 in Figure [4B](#)). The highlighted homo-oligomers of two-level symmetry in Figure [4B](#) have nearest-PDB TM-scores ranging from 0.25 to 0.52, with a median TM-score of 0.48; TM-scores for all seeds are plotted in Figure [S3C](#).

Another hierarchical design setting is to combine the function-scaffolding and the symmetric design tasks described above, as some functions are enhanced by repeti-

tion of a functional site; for example, when improving the strength of a binding interaction, multiple binding sites on a protein could synergize such that the overall binding avidity is greater than the sum of the individual affinities ([24](#)). This task requires two levels of hierarchy: the top level specifies symmetry while the bottom level specifies the side-chain atomic coordination constraint (Figure [5A](#)). With this corresponding program, we can generate designs in which an atomic-level constraint is enforced on multiple functional sites over the protein, the overall protein organization is constrained to be symmetric, and we can control the level of designed symmetry (Figures [5B](#) and [S4A–S4C](#)).

We lastly show that we can specify protein designs that have even deeper levels of hierarchy in their constraints (Figure [5C](#)) by designing protein assemblies that combine both symmetry and asymmetry. For example, we designed a protein complex composed of four units in which a pair of the chains are symmetric to each other (and each unit internally has two-fold symmetry) and where another pair of chains are symmetric to each other (and each unit also internally has two-fold symmetry), but the two pairs are asymmetric to each other (Figures [5C](#), [5D](#), [S4D](#), and [S4E](#)). Our high-level programming language readily enables us to control the complexity of the generated complexes such that, for example, one of the pairs consists of chains with three-fold symmetry (Figure [5E](#)) or that the complex consists of five chains (a pair of symmetric chains of two-fold symmetry asymmetrically complexed with a triple of symmetric chains of two-fold symmetry) (Figure [5F](#)). We find that our optimization procedure can produce designed structures consistent with all of these hierarchical specifications.

Related work

This paper is related to classical work that attempts to (i) classify a set of common sequence or structure motifs ([3](#), [4](#)) and (ii) manually combine these motifs to generate new proteins ([7–14](#)). More recently, deep-learning-based methods have increased the complexity of designable structures ([17](#), [20](#), [22](#)) and machine-learning-based generative models have shown increasingly sophisticated design capabilities. These include sequence-based Potts models and autoregressive language models for designing sequences ([25–27](#)), Markov Chain Monte Carlo algorithms combined with structure prediction for jointly designing sequences and structures ([17](#), [20](#), [22](#)), inverse folding models that use structural backbone coordinates to design sequences ([21](#), [23](#)), and concurrent work using diffusion models for designing protein backbones ([28](#), [29](#)). A key contribution of this study is to combine the modularity aspired to by classical methods with the power of modern generative models, in particular improvements in the accuracy and efficiency of language-model-based protein structure prediction ([16](#)).

用于生成蛋白质设计的高级编程语言

0.47到0.86，媒体TM分数为0.64；所有种子的TM分数绘制在图[S2C](#)。

为了增加我们对这些理想化结构对应于有效且可设计骨架的信心，我们观察到，通过使用ESM-IF1和ProteinMPNN模型进行逆折叠采样序列 ([21](#), [23](#)) 然后对序列样本进行结构预测可以重复恢复原始骨架几何形状（方法），在许多情况下子安斯特朗骨架原子RMSD。为了增加我们对成功的“往返旅行”通过逆折叠表示可设计骨架的信心，我们观察到高信心设计表示更好的往返旅行成功（图[3C](#)和[S2D](#)），并且能够采样低困惑度序列也表明往返旅行成功（图[3D](#)）。

超越单链对称性，我们还可以类似地设计多聚蛋白。我们在我们的编程语言中执行“单链”约束的概念，规定子树中的所有末端元素属于同一链（方法）；要设计多聚蛋白，我们只需要去除这个约束。图中提供了涉及4-mer到8-mer的多聚对称蛋白的示例[3E](#)。

层次约束

将我们的约束形式化为语法树，自然地使层次约束的规范化成为可能，这进一步促成了更复杂的蛋白质设计。作为初步演示，在我们高级语言的形式化指导下，我们设计了在两级层次结构中具有不同对称性的设计，其中较低层级的对称性在链内指定，而较高层级的对称性则在同源寡聚体的原聚物之间指定（图[4A](#)）。我们程序性地枚举了范围从具有2倍对称性的单位二聚体到具有4倍对称性的单位四聚体的范例（图[4B](#)、[S3A](#)和[S3B](#)）。在单链对称设计环境中，我们观察到通过逆折叠实现了成功的结构预测往返，并且高置信度的预测结构与低逆折叠困惑度均表明了往返成功（图[4C](#)和[4D](#)）。许多具有两级对称性的设计与PDB中的结构具有低整体相似度；例如，一个2倍对称的二聚体，其中对立的β薄片形成规则的棋盘模式（与PDB 3W38的最近-PDB TM-score为0.49；图中第1行和第1列[4B](#)）。图[4B](#) 中突出的两级对称的同源寡聚体的最近-PDB TM分数范围为0.25至0.52，中位数TM分数为0.48；所有种子的TM分数见图[S3C](#)。

另一个层次设计设置是结合上述描述的功能支架和对称设计任务，因为某些功能通过重复而得到了增强，

功能位点的重复；例如，在提高结合相互作用的强度时，蛋白质上的多个结合位点可以协同作用，使得整体的结合亲和力大于各个亲和力之和（[24](#)）。这个任务需要两个层次的层次结构：顶层指定对称性，而底层指定侧链原子协调约束（图[5A](#)）。通过这个相应的程序，我们可以生成设计，在这些设计中，对多个功能位点强加了原子级的约束，整个蛋白质的组织被约束为对称，并且我们可以控制设计的对称性程度（图[5B](#)和[S4A–S4C](#)）。

我们最后展示了我们可以指定具有更深层次层次结构的约束的蛋白质设计（图[5C](#)），通过设计结合对称性和非对称性的蛋白质组装。例如，我们设计了一个由四个单元组成的蛋白质复合物，其中一对链彼此对称（并且每个单元内部具有二重对称性），而另一对链也是彼此对称（并且每个单元同样内部具有二重对称性），但这两对之间是非对称的（图[5C](#), [5D](#), [S4D](#), 和[S4E](#)）。我们的高级编程语言使我们能够容易地控制生成的复合物的复杂性，例如，其中一对由具有三重对称性的链组成（图[5E](#)），或者该复合物由五条链组成（一对具有二重对称性的对称链与三条具有二重对称性的对称链非对称复合）（图[5F](#)）。我们发现我们的优化程序能够生成与所有这些层次结构规范一致的设计结构。

相关工作

本文与经典工作相关，该工作试图 (i) 对一组常见的序列或结构模式进行分类 ([3](#), [4](#)) 和 (ii) 手动组合这些模式以生成新的蛋白质 ([7–14](#))。最近，基于深度学习的方法增加了可设计结构的复杂性 ([17](#), [20](#), [22](#))，而基于机器学习的生成模型显示出越来越复杂的设计能力。这些包括基于序列的波茨模型和自回归语言模型用于设计序列 ([25–27](#))，将马尔可夫链蒙特卡罗算法与结构预测相结合，以联合设计序列和结构 ([17](#), [20](#), [22](#))，使用结构骨架坐标设计序列的逆折叠模型 ([21](#), [23](#))，以及使用扩散模型设计蛋白质骨架的同时工作 ([28](#), [29](#))。本研究的一个关键贡献是将经典方法所期望的模块化与现代生成模型的强大能力相结合，尤其是在基于语言模型的蛋白质结构预测的准确性和效率方面的改进 ([16](#))。

A high-level programming language for generative protein design

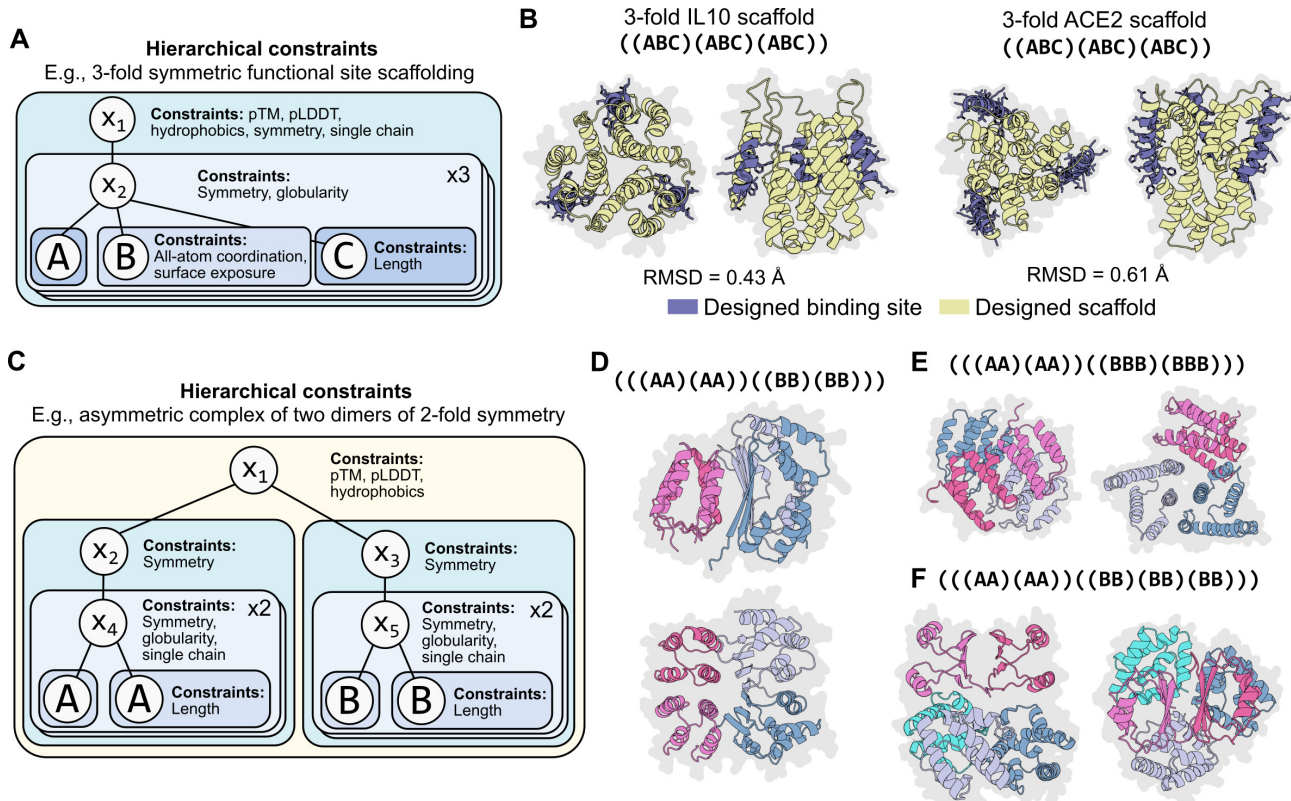


Figure 5. Programming complex hierarchical constraints. (A) A graphical representation of a program for scaffolding three functional sites in which those sites have a 3-fold symmetry. (B) Example 3-fold symmetric scaffolds for the IL10 and ACE2 binding sites that achieve sub-angstrom RMSD averaged across the three sites. (C) A graphical representation of a program that specifies an asymmetric protein complex consisting of two pairs of chains. Each pair is constrained to have 2-fold symmetry between the constituent chains. Furthermore, each constituent chain itself has 2-fold symmetry. (D) A generated protein structure as specified by the program depicted in (C). Discrete chains are indicated by different colors. (E) A generated protein structure as specified by the program depicted in (C) except where one of the pairs has constituent chains that have three-fold symmetry. Discrete chains are indicated by different colors. (F) A generated protein structure as specified by the program depicted in (C) except where one of the pairs is replaced with a symmetric trimer (where each constituent chain in the trimer has two-fold symmetry). Discrete chains are indicated by different colors.

用于生成蛋白质设计的高级编程语言

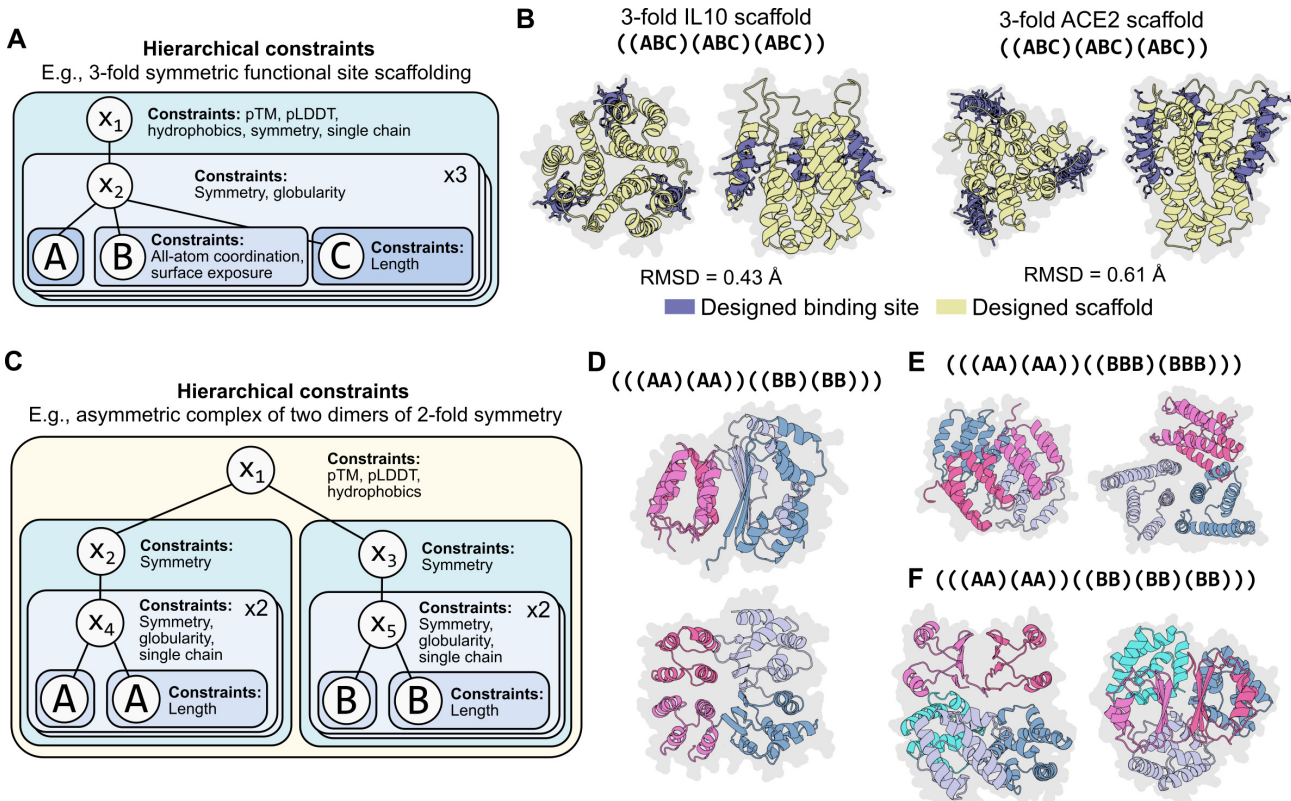


图5. 编程复杂的层次约束。(A) 一个程序的图形表示，该程序用于在具有三重对称的三个功能位点之间构建支架。(B) IL10和ACE2结合位点的例子，具有3倍对称的支架，达到在三个位点之间的亚埃级RMSD的平均值。(C) 一个图形表示的程序，指定一个不对称的蛋白质复合物，由两对链组成。每对链之间的成分链被约束为具有2倍对称。此外，每个成分链本身也具有2倍对称。(D) 根据图 (C) 所示程序生成的蛋白质结构。不同颜色表示离散链。(E) 根据图 (C) 所示程序生成的蛋白质结构，唯一的例外是其中一对的成分链具有三重对称。不同颜色表示离散链。(F) 根据图 (C) 所示程序生成的蛋白质结构，唯一的例外是其中一对被一个对称三聚体替换（其中三聚体中的每个成分链具有二重对称）。不同颜色表示离散链。

A high-level programming language for generative protein design

Discussion

In this study, we show that generative artificial intelligence enables high-level programmability at a new level of combinatorial complexity. We propose a programming language that can express high-level programs for the design of proteins at diverse biological scales, including atomic-level coordinates, secondary structure, and high-level symmetries within single chains and the units of self-assembling multi-chain complexes. We show that programs written in the abstract language can be compiled into an energy function and that the corresponding generative model is capable of fulfilling complex constraints within an overall coherent structure.

We demonstrate programs of increasing levels of complexity, including the design of homo-oligomers with two levels of symmetry, symmetric functional scaffolds, and asymmetric complexes of subunits that themselves have two levels of symmetry. The approach reveals a large space of idealized protein designs created from top-down design principles. Especially as the complexity of the constraints increases, many of the corresponding designs are highly idealized, analogous to the regularity of artificially created machines and systems.

Our computational results using two independent inverse folding methods suggest that the generated structures are designable, since inverse folding models have demonstrated high experimental success rates (23). We are also obtaining data to experimentally validate the designs.

More broadly, the formalization offered by a high-level programming language enables logical design principles to be applied to protein design as in other fields of engineering. This has been especially challenging in biology due to the way that the amino acid sequence opaquely encodes the structure and function. As protein design moves toward the engineering of more complex functions, we anticipate that such a system will become increasingly useful.

Acknowledgements

We would like to thank Halil Akin, Adam Lerer, Wenting Lu, Yaniv Shmueli, Robert Verkuil, and Zhongkai Zhu for technical help, feedback, and discussions that helped shape this project. We thank Laurens van der Maaten, Ammar Rizvi, Jon Shepard, and Joe Spisak for program support.

References

[1] Cyrus Levinthal. Are there pathways for protein folding? *Extrait du Journal de Chimie Physique*, 65(1):44, 1968.

[2] A M Lesk and G D Rose. Folding units in globular

proteins. *Proc. Natl. Acad. Sci. U. S. A.*, 78(7):4304–4308, July 1981.

[3] William R Taylor. A ‘periodic table’ for protein structures. *Nature*, 416(6881):657–660, April 2002.

[4] Craig O Mackenzie, Jianfu Zhou, and Gevorg Grigoryan. Tertiary alphabet for the observable protein structural universe. *Proc. Natl. Acad. Sci. U. S. A.*, 113(47):E7438–E7447, November 2016.

[5] W Kabsch and C Sander. On the use of sequence homologies to predict protein structure: Identical pentapeptides can have completely different conformations. *Proc. Natl. Acad. Sci. U. S. A.*, 81(4):1075–1078, February 1984.

[6] D L Minor, Jr and P S Kim. Context-dependent secondary structure formation of a designed protein sequence. *Nature*, 380(6576):730–734, April 1996.

[7] P B Harbury, J J Plecs, B Tidor, T Alber, and P S Kim. High-resolution protein design with backbone freedom. *Science*, 282(5393):1462–1467, November 1998.

[8] Brian Kuhlman, Gautam Dantas, Gregory C Ireton, Gabriele Varani, Barry L Stoddard, and David Baker. Design of a novel globular protein fold with atomic-level accuracy. *Science*, 302(5649):1364–1368, November 2003.

[9] Hui-Hsu Gavin Tsai, Chung-Jung Tsai, Buyong Ma, and Ruth Nussinov. In silico protein design by combinatorial assembly of protein building blocks. *Protein Sci.*, 13(10):2753–2765, October 2004.

[10] Nobuyasu Koga, Rie Tatsumi-Koga, Gaohua Liu, Rong Xiao, Thomas B Acton, Gaetano T Montelione, and David Baker. Principles for designing ideal protein structures. *Nature*, 491(7423):222–227, November 2012.

[11] Che Yang, Fabian Sesterhenn, Jaume Bonet, Eva A van Aalen, Leo Scheller, Luciano A Abriata, Johannes T Cramer, Xiaolin Wen, Stéphane Rosset, Sandrine Georgeon, Theodore Jardetzky, Thomas Krey, Martin Fussenegger, Maarten Merkkx, and Bruno E Correia. Bottom-up de novo design of functional proteins with complex structural features. *Nat. Chem. Biol.*, 17(4):492–500, April 2021.

[12] H Zhao, L Giver, Z Shao, J A Affholter, and F H Arnold. Molecular evolution by staggered extension process (StEP) in vitro recombination. *Nat. Biotechnol.*, 16(3):258–261, March 1998.

用于生成蛋白质设计的高级编程语言

讨论

在本研究中，我们表明生成性人工智能能够在新的组合复杂性层面上实现高级可编程性。我们提出了一种编程语言，可以表达用于蛋白质设计的高级程序，这些设计可在不同的生物尺度上进行，包括原子级坐标、二级结构和单链及自组装多链复合物单元内的高级对称性。我们展示了用抽象语言编写的程序可以被编译成能量函数，并且相应的生成模型能够在整体一致结构中满足复杂约束。

我们展示了复杂性水平逐渐增加的程序，包括设计具有两级对称性的同聚物、对称功能支架以及自身具有两级对称性的子单位的不对称复合物。这一方法揭示了通过自上而下的设计原则创建的理想化蛋白质设计的大型空间。特别是随着约束复杂性增加，许多相应的设计高度理想化，类似于人造机器和系统的规则性。

我们使用两种独立的逆折叠方法进行的计算结果表明，生成的结构是可设计的，因为逆折叠模型已证明具有高实验成功率（23）。我们还正在获得数据以实验验证这些设计。

更广泛地说，高级编程语言所提供的形式化使得逻辑设计原则能够被应用于蛋白质设计，如同工程学的其他领域。这在生物学中尤为具有挑战性，因为氨基酸序列不透明地编码了结构和功能。随着蛋白质设计朝着更复杂功能的工程化发展，我们预计这样的系统将变得愈加有用。

致谢

我们感谢Halil Akin、亚当·勒雷、Wenting Lu、Yaniv Shmueli、罗伯特·维尔库尔和Zhongkai Zhu在技术支持、反馈及讨论方面的帮助，这些都对项目的形成起到了关键作用。我们还感谢Laurens van der Maaten、Ammar Rizvi、Jon Shepard和Joe Spisak的程序支持。

参考文献

[1] 赛鲁斯·莱文索尔。蛋白质折叠是否存在路径？摘自《物理化学杂志》，65(1):44, 1968。

[2] A M 莱斯克 和 G D 罗斯. 球状蛋白质中的折叠单元

《美国国家科学院院刊》，78(7): 4304–4308, 1981年7月。

[3] 威廉·R·泰勒。一种用于蛋白质结构的‘周期表’。自然，416(6881):657–660, 2002年4月。

[4] Craig O Mackenzie、周建福和格韦戈尔·格里戈良。可观测蛋白质结构宇宙的三级字母表。美国国家科学院院刊，113(47):E7438–E7447, 2016年11月。

[5] W·卡布什和C·桑德。利用序列同源性预测蛋白质结构：相同的肽链可以具有完全不同的构象。美国国家科学院院刊，81(4):1075–1078, 1984年2月。

[6] D·L·小_MINOR和P·S·金。设计的蛋白质序列的上下文相关次级结构形成。自然，380(6576):730–734, 1996年4月。

[7] P·B·哈巴里，J·J·普莱克斯，B·蒂多尔，T·阿尔伯，与P·S·金。自由骨架的高分辨率蛋白质设计。科学，282(5393): 1462–1467, 1998年11月。

[8]布赖恩·库尔曼，高特姆·丹塔斯，格雷戈里·C·艾尔顿，加布里埃尔·瓦拉尼，巴里·L·斯托达德，与David Baker。以原子级精度设计新的球状蛋白折叠。科学，302(5649): 1364–1368, 2003年11月。

[9] 蔡辉旭，蔡忠正，马不勇，与鲁思·努辛诺夫。通过蛋白质构建块的组合组装进行计算蛋白设计。蛋白质科学，13(10): 2753–2765, 2004年10月。

[10] 小笠原信康，辰巳理惠，刘高华，肖荣，托马斯·B·阿克顿，盖塔诺·T·蒙特利昂，与David Baker。设计理想蛋白质结构的原则。自然，491(7423): 222–227, 2012年11月。

[11] 杨喆，法比安·塞斯特伦、豪梅·博内特、艾娃·A·范阿伦、里奥·谢勒、卢西亚诺·A·阿布里亚塔、约翰内斯·T·克雷默、温晓琳、斯特凡·罗塞、桑德琳·乔治昂、西奥多·贾尔德兹基、托马斯·克雷、马丁·富森厄格、马尔滕·梅克斯和布鲁诺·E·科雷亚。自下而上的新型设计具有复杂结构特征的功能性蛋白。Nat. Chem. Biol., 17(4): 492–500, 2021年4月。

[12] 赵华，L·吉弗、Z·邵、J·A·阿弗霍尔特和F·H·阿诺德。通过交错扩展过程 (StEP) 进行的体外重组的分子进化。Nat. Biotechnol., 16(3): 258–261, 1998年3月。

A high-level programming language for generative protein design

[13] Christopher A Voigt, Carlos Martinez, Zhen-Gang Wang, Stephen L Mayo, and Frances H Arnold. Protein building blocks preserved by recombination. *Nat. Struct. Biol.*, 9(7):553–558, July 2002.

[14] Marc Ostermeier. Engineering allosteric protein switches by domain insertion. *Protein Eng. Des. Sel.*, 18(8):359–364, August 2005.

[15] Alexander Rives, Joshua Meier, Tom Sercu, Siddharth Goyal, Zeming Lin, Jason Liu, Demi Guo, Myle Ott, C Lawrence Zitnick, Jerry Ma, and Rob Fergus. Biological structure and function emerge from scaling unsupervised learning to 250 million protein sequences. *Proc. Natl. Acad. Sci. U. S. A.*, 118(15):e2016239118, April 2021.

[16] Zeming Lin, Halil Akin, Roshan Rao, Brian Hie, Zhongkai Zhu, Wenting Lu, Nikita Smetanin, Robert Verkuil, Ori Kabeli, Yaniv Shmueli, Allan dos Santos Costa, Maryam Fazel-Zarandi, Tom Sercu, Salvatore Candido, and Alexander Rives. Evolutionary-scale prediction of atomic level protein structure with a language model. *bioRxiv*, 2022. doi: 10.1101/2022.07.20.500902.

[17] Ivan Anishchenko, Samuel J Pellock, Tamuka M Chidyausiku, Theresa A Ramelot, Sergey Ovchinnikov, Jingzhou Hao, Khushboo Bafna, Christoffer Norn, Alex Kang, Asim K Bera, Frank DiMaio, Lauren Carter, Cameron M Chow, Gaetano T Montelione, and David Baker. De novo protein design by deep network hallucination. *Nature*, 600(7889):547–552, December 2021.

[18] John Jumper, Richard Evans, Alexander Pritzel, Tim Green, Michael Figurnov, Olaf Ronneberger, Kathryn Tunyasuvunakool, Russ Bates, Augustin Žídek, Anna Potapenko, Alex Bridgland, Clemens Meyer, Simon A A Kohl, Andrew J Ballard, Andrew Cowie, Bernardino Romera-Paredes, Stanislav Nikolov, Rishub Jain, Jonas Adler, Trevor Back, Stig Petersen, David Reiman, Ellen Clancy, Michal Zielinski, Martin Steinegger, Michalina Pacholska, Tamas Berghammer, Sebastian Bodenstern, David Silver, Oriol Vinyals, Andrew W Senior, Koray Kavukcuoglu, Pushmeet Kohli, and Demis Hassabis. Highly accurate protein structure prediction with AlphaFold. *Nature*, 596(7873):583–589, August 2021.

[19] Po-Ssu Huang, Scott E Boyken, and David Baker. The coming of age of de novo protein design. *Nature*, 537(7620):320–327, September 2016.

[20] Jue Wang, Sidney Lisanza, David Juergens, Doug Tischer, Joseph L Watson, Karla M Castro, Robert Ragotte, Amijai Saragovi, Lukas F Milles, Minkyung

Baek, Ivan Anishchenko, Wei Yang, Derrick R Hicks, Marc Expòsit, Thomas Schlichthaerle, Jung-Ho Chun, Justas Dauparas, Nathaniel Bennett, Basile I M Wicky, Andrew Muenks, Frank DiMaio, Bruno Correia, Sergey Ovchinnikov, and David Baker. Scaffolding protein functional sites using deep learning. *Science*, 377(6604):387–394, July 2022.

[21] Chloe Hsu, Robert Verkuil, Jason Liu, Zeming Lin, Brian Hie, Tom Sercu, Adam Lerer, and Alexander Rives. Learning inverse folding from millions of predicted structures. In *Proceedings of the 39th International Conference on Machine Learning*, pages 8946–8970, 2022.

[22] B I M Wicky, L F Milles, A Courbet, R J Ragotte, J Dauparas, E Kinfu, S Tipps, R D Kibler, M Baek, F DiMaio, X Li, L Carter, A Kang, H Nguyen, A K Bera, and D Baker. Hallucinating symmetric protein assemblies. *Science*, 378(6615):56–61, October 2022.

[23] J Dauparas, I Anishchenko, N Bennett, H Bai, R J Ragotte, L F Milles, B I M Wicky, A Courbet, R J de Haas, N Bethel, P J Y Leung, T F Huddy, S Pellock, D Tischer, F Chan, B Koepnick, H Nguyen, A Kang, B Sankaran, A K Bera, N P King, and D Baker. Robust deep learning-based protein sequence design using ProteinMPNN. *Science*, 378(6615):49–56, October 2022.

[24] Georges Vauquelin and Steven J Charlton. Exploring avidity: understanding the potential gains in functional affinity and target residence time of bivalent and heterobivalent ligands. *Br. J. Pharmacol.*, 168(8):1771–1785, April 2013.

[25] William P Russ, Matteo Figliuzzi, Christian Stocker, Pierre Barrat-Charlaix, Michael Socolich, Peter Kast, Donald Hilvert, Remi Monasson, Simona Cocco, Martin Weigt, and Rama Ranganathan. An evolution-based model for designing chorisimate mutase enzymes. *Science*, 369(6502):440–445, July 2020.

[26] Ali Madani, Ben Krause, Eric R. Greene, Subu Subramanian, Benjamin P. Mohr, James M. Holton, Jose Luis Olmos, Caiming Xiong, Zachary Z. Sun, Richard Socher, James S. Fraser, and Nikhil Naik. Deep neural language modeling enables functional protein generation across families. *bioRxiv*, 2021. doi: 10.1101/2021.07.18.452833.

[27] Noelia Ferruz, Steffen Schmidt, and Birte Höcker. ProtGPT2 is a deep unsupervised language model for protein design. *Nat. Commun.*, 13(1):4348, July 2022.

[28] John Ingraham, Max Baranov, Zak Costello, Vincent Frappier, Ahmed Ismail, Shan Tie, Wujie Wang, Vincent Xue, Fritz Obermeyer, Andrew Beam, and Gevorg

.CC-BY-NC-ND 4.0国际许可证可在（未经同行评审的）下获得，由作者/资金提供者提供，已授予bioRxiv永久展示该预印本的许可。本预印本的版权所有者为本版本，于2022年12月22日发布。; <https://doi.org/10.1101/2022.12.21.521526> doi: bioRxiv预印本

一种用于生成蛋白质设计的高级编程语言

[13] Christopher A Voigt, Carlos Martinez, Zhen-Gang Wang, Stephen L Mayo, 和 Frances H Arnold. 通过重组保留的蛋白质构建模块. 自然结构生物学, 9(7):553–558, 2002年7月.

[14]Marc Ostermeier. 通过领域插入工程化别构蛋白开关。蛋白质工程设计选择, 18(8):359–364, 2005年8月。

[15] Alexander Rives, Joshua Meier, Tom Sercu, Siddharth Goyal, Zeming Lin, Jason Liu, Demi Guo, Myle Ott, C Lawrence Zitnick, Jerry Ma和Rob Fergus. 生物结构和功能通过将无监督学习扩展到2.5亿个蛋白质序列而产生。美国国家科学院院刊, 118(15):e2016239118, 2021年4月。

[16] Zeming Lin, Halil Akin, Roshan Rao, Brian Hie, Zhongkai Zhu, Wenting Lu, Nikita Smetanin, 罗伯特·维尔库尔, Ori Kabeli, Yaniv Shmueli, Allan dos Santos Costa, Maryam Fazel-Zarandi, Tom Sercu, Salvatore Candido和Alexander Rives. 使用语言模型进行原子级蛋白质结构的进化规模预测。bioRxiv, 2022年. doi: 10.1101/2022.07.20.500902。

[17] 伊万·阿尼申科, 塞缪尔·J·佩洛克, 塔穆卡·M·奇迪亚乌西库, 特雷莎·A·拉梅洛特, 谢尔盖·奥夫钦尼科夫, 景洲·郝, 库什布·巴夫纳, 克里斯托弗·诺恩, 亚历克斯·康, 阿西姆·K·贝拉, 弗兰克·迪迈奥, 劳伦·卡特, 卡梅伦·M·乔, 盖塔诺·T·蒙特利昂, 以及大卫·贝克。通过深度网络幻觉进行的新生蛋白质设计。《自然》, 600(7889): 547–552, 2021年12月。

[18]约翰·贾姆珀, 理查德·埃文斯, 亚历山大·普里策尔, 蒂姆·格林, 迈克尔·菲古尔诺夫, 奥拉夫·罗内贝格, 凯瑟琳·图亚苏瓦那库尔, 拉斯·贝茨, 奥古斯丁·兹伊德克, 安娜·波塔彭科, 亚历克斯·布里奇兰, 克莱门斯·迈耶, 西蒙·A·A·科尔, 安德鲁·J·巴拉德, 安德鲁·考威, 贝尔纳迪诺·罗梅拉·帕雷德斯, 斯坦尼斯拉夫·尼科洛夫, 里舒布·贾因, 乔纳斯·阿德勒, 特雷弗·巴克, 斯蒂格·彼得森, 大卫·雷曼, 艾伦·克兰西, 米哈乌·齐林斯基, 马丁·斯坦尼格, 米哈利娜·帕霍尔斯卡, 塔马斯·贝尔哈默, 塞巴斯蒂安·博登斯坦, 大卫·西尔弗, 奥里奥尔·维尼亚尔斯, 安德鲁·W·西尼尔, 科雷·卡武克约格, 普什米特·科赫利, 以及德米斯·哈萨比斯。使用AlphaFold进行的高精度蛋白质结构预测。《自然》, 596(7873): 583–589, 2021年8月。

[19] 黄博思, Scott E Boyken, 和David Baker. 新生蛋白质设计的成熟. 自然, 537 (7620):320–327, 2016年9月。

[20] 王珏, 西德尼·利赞扎, 大卫·朱根斯, 道格·蒂舍, 约瑟夫·L·沃森, 卡拉·M·卡斯特罗, 罗伯特·拉戈特, 阿米贾伊·萨拉戈维, 卢卡斯·F·米勒斯, 闵京权

白冈京, 伊万·阿尼申科, 杨威, 德里克·R·希克斯, 马尔克·埃克斯波西特, 托马斯·施利希特哈尔勒, 正浩·全, 尤斯塔斯·道帕拉斯, 内森尼尔·本内特, 巴西尔·I·M·维基, 安德鲁·穆恩克斯, 弗兰克·迪迈奥, 布鲁诺·科雷亚, 谢尔盖·奥夫奇尼科夫, 以及大卫·贝克。使用深度学习构建支架蛋白功能位点。《科学》, 377(6604): 387–394, 2022年7月。

[21] 徐克洛伊, 罗伯特·维尔库尔, 刘杰森, 林泽铭, Brian Hie, Tom Sercu, 亚当·勒雷, 和 Alexander Rives. 从数百万个预测结构中学习逆折叠. 在第39届国际机器学习会议论文集中, 页码 8946–8970, 2022.

[22] B I M Wicky, L F 米勒斯, A·库尔贝, R·J·拉戈特, J Dauparas, E Kinfu, S Tipps, R D Kibler, M Baek, F DiMaio, X Li, L Carter, A Kang, H Nguyen, A K Bera, 和D Baker. 幻觉对称蛋白组装. 科学, 378(6615):56–61, 2022年10月。

[23] J·道帕拉斯, I·阿尼申科, N·贝内特, H·白, R·J·拉戈特, L·F·米勒斯, B·I·M·维基, A·库尔贝, R·J·德·哈斯, N·贝塞尔, P·J·Y·梁, T·F·哈迪, S·佩洛克, D·提舍尔, F·陈, B·科普尼克, H·阮, A·康, B·桑卡兰, A·K·贝拉, N·P·金, 和D·贝克。基于深度学习的ProteinMPNN蛋白质序列设计。科学, 378(6615):49–56, 2022年10月。

[24] 乔治·沃克林和史蒂文·J·查尔顿。探讨亲和力：理解二价和杂二价配体在功能亲和力和目标停留时间上的潜在增益。《英国药理学杂志》, 168(8): 1771–1785, 2013年4月。

[25] 威廉·P·拉斯, 马泰奥·菲格利乌齐, 克里斯蒂安·斯托克, 皮埃尔·巴拉特-查拉夫, 迈克尔·索科利奇, 彼得·卡斯特, 唐纳德·希尔维特, 雷米·莫纳松, 西蒙娜·科科, 马丁·维格特, 和拉马·兰卡纳坦。基于进化的模型设计香草酰胺变位酶。科学, 369(6502):440–445, 2020年7月。

[26]Ali Madani, Ben Krause, Eric R. Greene, Subu Subramanian, Benjamin P. Mohr, James M. Holton, Jose Luis Olmos, Caiming Xiong, Zachary Z. Sun, Richard Socher, James S. Fraser, 以及Nikhil Naik. 深度神经语言建模使蛋白质在不同家族中得以功能生成. bioRxiv, 2021. doi: 10.1101/2021.07.18.452833.

[27] Noelia Ferruz, Steffen Schmidt, 以及Birte Höcker. ProtGPT2是一种用于蛋白质设计的深度无监督语言模型. 自然通讯, 13(1):4348, 2022年7月。

[28] John Ingraham, Max Baranov, Zak Costello, Vincent Frappier, Ahmed Ismail, Shan Tie, Wujie Wang, Vincent Xue, Fritz Obermeyer, Andrew Beam, 以及Gevorg

10

10

A high-level programming language for generative protein design

Grigoryan. Illuminating protein space with a programmable generative model. *bioRxiv*, 2022. doi: 10.1101/2022.12.01.518682.

[29] Joseph L. Watson, David Juergens, Nathaniel R. Bennett, Brian L. Trippe, Jason Yim, Helen E. Eisenach, Woody Ahern, Andrew J. Borst, Robert J. Ragotte, Lukas F. Milles, Basile I. M. Wicky, Nikita Hanikel, Samuel J. Pellock, Alexis Courbet, William Sheffler, Jue Wang, Preetham Venkatesh, Isaac Sappington, Susana Vázquez Torres, Anna Lauko, Valentin De Bortoli, Emile Mathieu, Regina Barzilay, Tommi S. Jaakkola, Frank DiMaio, Minkyung Baek, and David Baker. Broadly applicable and accurate protein design by integrating structure prediction networks and diffusion generative models. *bioRxiv*, 2022. doi: 10.1101/2022.12.09.519842.

[30] A Shrake and J A Rupley. Environment and exposure to solvent of protein atoms. lysozyme and insulin. *J. Mol. Biol.*, 79(2):351–371, September 1973.

[31] Patrick Kunzmann and Kay Hamacher. Biotite: a unifying open source computational biology framework in python. *BMC Bioinformatics*, 19(1):346, 2018.

[32] G Labesse, N Colloc’h, J Pothier, and J P Mornon. P-SEA: a new efficient assignment of secondary structure from C alpha trace of proteins. *Comput. Appl. Biosci.*, 13(3):291–295, June 1997.

[33] W. Kabsch. A solution for the best rotation to relate two sets of vectors. *Acta Crystallographica Section A*, 32(5):922–923, 1976. doi: <https://doi.org/10.1107/S0567739476001873>. URL <https://onlinelibrary.wiley.com/doi/abs/10.1107/S0567739476001873>.

[34] Sung Il Yoon, Brandi C Jones, Naomi J Logsdon, and Mark R Walter. Same structure, different function crystal structure of the Epstein-Barr virus IL-10 bound to the soluble IL-10R1 chain. *Structure*, 13(4):551–564, April 2005.

[35] Jun Lan, Jiwan Ge, Jinfang Yu, Sisi Shan, Huan Zhou, Shilong Fan, Qi Zhang, Xuanling Shi, Qisheng Wang, Linqi Zhang, and Xinquan Wang. Structure of the SARS-CoV-2 spike receptor-binding domain bound to the ACE2 receptor. *Nature*, 581(7807):215–220, May 2020.

[36] G Szakonyi, J M Guthridge, D Li, K Young, V M Holers, and X S Chen. Structure of complement receptor 2 in complex with its C3d ligand. *Science*, 292(5522):1725–1728, June 2001.

[37] Nicole L Kallewaard, Davide Corti, Patrick J Collins, Ursula Neu, Josephine M McAuliffe, Ebony Benjamin, Leslie Wachter-Rosati, Frances J Palmer-Hill, Andy Q Yuan, Philip A Walker, Matthias K Vorlaender, Siro Bianchi, Barbara Guarino, Anna De Marco, Fabrizia Vanzetta, Gloria Agatic, Mathilde Foglierini, Debora Pinna, Blanca Fernandez-Rodriguez, Alexander Fruehwirth, Chiara Silacci, Roksana W Ogradowicz, Stephen R Martin, Federica Sallusto, Joann A Suzich, Antonio Lanzavecchia, Qing Zhu, Steven J Gamblin, and John J Skehel. Structure and function analysis of an antibody recognizing all influenza a subtypes. *Cell*, 166(3):596–608, July 2016.

[38] Kathryn Westendorf, Stefanie Žentelis, Lingshu Wang, Denisa Foster, Peter Vaillancourt, Matthew Wiggin, Erica Lovett, Robin van der Lee, Jörg Hendle, Anna Pustilnik, J Michael Sauder, Lucas Kraft, Yuri Hwang, Robert W Siegel, Jinbiao Chen, Beverly A Heinz, Richard E Higgs, Nicole L Kallewaard, Kevin Jepson, Rodrigo Goya, Maia A Smith, David W Collins, Davide Pellacani, Ping Xiang, Valentine de Puyraimond, Marketa Ricicova, Lindsay Devorkin, Caitlin Pritchard, Aoise O’Neill, Kush Dalal, Pankaj Panwar, Harveer Dhupar, Fabian A Garces, Courtney A Cohen, John M Dye, Kathleen E Huie, Catherine V Badger, Darwyn Kobasa, Jonathan Audet, Joshua J Freitas, Saleema Hassanali, Ina Hughes, Luis Munoz, Holly C Palma, Bharathi Ramamurthy, Robert W Cross, Thomas W Geisbert, Vineet Menachery, Kumari Lokugamage, Viktoriya Borisevich, Iliana Lanz, Lisa Anderson, Payal Sipahimalani, Kizzmekia S Corbett, Eun Sung Yang, Yi Zhang, Wei Shi, Tongqing Zhou, Misook Choe, John Misasi, Peter D Kwong, Nancy J Sullivan, Barney S Graham, Tara L Fernandez, Carl L Hansen, Ester Falconer, John R Mascola, Bryan E Jones, and Bryan C Barnhart. LY-CoV1404 (bebtelovimab) potentially neutralizes SARS-CoV-2 variants. *Cell Rep.*, 39(7):110812, May 2022.

[39] H M Berman, J Westbrook, Z Feng, G Gilliland, T N Bhat, H Weissig, I N Shindyalov, and P E Bourne. The protein data bank. *Nucleic Acids Res.*, 28(1):235–242, January 2000.

[40] Yang Zhang and Jeffrey Skolnick. TM-align: a protein structure alignment algorithm based on the TM-score. *Nucleic Acids Res.*, 33(7):2302–2309, April 2005.

一种用于生成蛋白质设计的高级编程语言

Grigoryan. 用可编程生成模型照亮蛋白质空间。bioRxiv, 2022. doi: 10.1101/2022.12.01.518682.

[29] Joseph L. Watson, David Juergens, Nathaniel R. Bennett, Brian L. Trippe, Jason Yim, Helen E. Eisenach, Woody Ahern, Andrew J. Borst, Robert J. Ragotte, Lukas F. Milles, Basile I. M. Wicky, Nikita Hanikel, Samuel J. Pellock, Alexis Courbet, William Sheffler, Jue Wang, Preetham Venkatesh, Isaac Sappington, Susana Vázquez Torres, Anna Lauko, Valentin De Bortoli, Emile Mathieu, Regina Barzilay, Tommi S. Jaakkola, Frank DiMaio, Minkyung Baek 和 David Baker。通过整合结构预测网络和扩散生成模型，实现广泛适用和高精度的蛋白质设计。bioRxiv, 2022. doi: 10.1101/2022.12.09.519842。

[30] A Shrake 和 J A Rupley. 蛋白质原子对溶剂的环境和暴露：溶菌酶和胰岛素. *J. Mol. Biol.*, 79(2):351–371, 1973年9月。

[31] Patrick Kunzmann 和 Kay Hamacher. biotite: 一个统一的开源计算生物学框架，采用 Python 语言. *BMC Bioinformatics*, 19(1):346, 2018.

[32] G Labesse, N Colloc’h, J Pothier 和 J P Mornon. P-SEA：一种新的高效二级结构分配方法，基于蛋白质的 C α 轨迹. *Comput. Appl. Biosci.*, 13(3):291–295, 1997年6月。

[33]W. Kabsch. 解决两个向量集之间最佳旋转的问题. *Acta Crystallographica Section A*,32(5):922–923, 1976.doi: <https://doi.org/10.1107/S0567739476001873>.URL <https://onlinelibrary.wiley.com/doi/abs/10.1107/S0567739476001873>.

[34] Sung Il Yoon, Brandi C Jones, Naomi J Logsdon 和 Mark R Walter. 相同结构，不同功能的 Epstein-Barr 病毒 IL-10 与可溶 IL-10R1 链结合的晶体结构. *Structure*, 13(4): 551–564, 2005 年 4 月。

[35] Jun Lan, Jiwan Ge, Jinfang Yu, Sisi Shan, Huan Zhou, Shilong Fan, Qi Zhang, Xuanling Shi, Qisheng Wang, Linqi Zhang 和 Xinquan Wang. SARS-CoV-2 刺突受体结合域与 ACE2 受体结合的结构. *自然*, 581(7807): 215–220, 2020 年 5 月。

[36] G Szakonyi, J M Guthridge, D Li, K Young, V M Holers, 和 X S Chen. 补体受体 2 与其 C3d 配体的复合物结构. *科学*, 292 (5522): 1725–1728, 2001 年 6 月。

[37] Nicole L Kallewaard, Davide Corti, Patrick J Collins, Ursula Neu, Josephine M McAuliffe, Ebony Benjamin, Leslie Wachter-Rosati, Frances J Palmer-Hill, Andy Q Yuan, Philip A Walker, Matthias K Vorlaender, Siro Bianchi, Barbara Guarino, Anna De Marco, Fabrizia Vanzetta, Gloria Agatic, Mathilde Foglierini, Debora Pinna, Blanca Fernandez-Rodriguez, Alexander Fruehwirth, Chiara Silacci, Roksana W Ogradowicz, Stephen R Martin, Federica Sallusto, Joann A Suzich, Antonio Lanzavecchia, Qing Zhu, Steven J Gamblin, 和 John J Skehel. 识别所有流感 A 亚型的抗体的结构与功能分析. *细胞*, 166(3): 596–608, 2016 年 7 月。

[38] Kathryn Westendorf, Stefanie Zentelis, Lingshu Wang, Denisa Foster, Peter Vaillancourt, Matthew Wiggin, Erica Lovett, Robin van der Lee, J’org Hendle, Anna Pustilnik, J Michael Sauder, Lucas Kraft, Yuri Hwang, Robert W Siegel, Jinbiao Chen, Beverly A Heinz, Richard E Higgs, Nicole L Kallewaard, Kevin Jepson, Rodrigo Goya, Maia A Smith, David W Collins, Davide Pellacani, Ping Xiang, Valentine de Puyraimond, Marketa Ricicova, Lindsay Devorkin, Caitlin Pritchard, Aoise O’Neill, Kush Dalal, Pankaj Panwar, Harveer Dhupar, Fabian A Garces, Courtney A Cohen, John M Dye, Kathleen E Huie, Catherine V Badger, Darwyn Kobasa, Jonathan Audet, Joshua J Freitas, Saleema Hassanali, Ina Hughes, Luis Munoz, Holly C Palma, Bharathi Ramamurthy, Robert W Cross, Thomas W Geisbert, Vineet Menachery, Kumari Lokugamage, Viktoriya Borisevich, Iliana Lanz, Lisa Anderson, Payal Sipahimalani, Kizzmekia S Corbett, Eun Sung Yang, Yi Zhang, Wei Shi, Tongqing Zhou, Misook Choe, John Misasi, Peter D Kwong, Nancy J Sullivan, Barney S Graham, Tara L Fernandez, Carl L Hansen, Ester Falconer, John R Mascola, Bryan E Jones 和 Bryan C Barnhart. LY-CoV1404 (bebtelovimab) 有效中和 SARS-CoV-2 变种. *Cell Rep.*, 39(7): 110812, 2022 年 5 月。

[39] H M Berman, J Westbrook, Z Feng, G Gilliland, T N Bhat, H Weissig, I N Shindyalov 和 P E Bourne. 蛋白质数据银行. *核酸研究*, 28(1):235–242, 2000 年 1 月。

[40] 杨章 和 杰弗里·斯科尔尼克. TM-align：一种基于 TM-score 的蛋白质结构对齐算法. *核酸研究*, 33(7):2302–2309, 2005 年 4 月。

A high-level programming language for generative protein design

A. Methods

A.1. High-level programming language and energy-based optimization

A program in our language is fully specified by (1) a syntax tree and (2) a set of constraints. This program compiles to an energy function, which is used to guide black-box optimization of a protein sequence while also leveraging its predicted structure.

A.1.1. SYNTAX TREE

The syntax tree consists of nonterminal symbols, which we denote as x_i , as well as terminal symbols, which in our examples we denote as uppercase alphabetic characters such as A, B, C , etc. Each terminal symbol defines a unique protein sequence. The nonterminal symbol x_1 is designated as the special start symbol; all programs must have x_1 . Additional nonterminal symbols are used to define hierarchical complexity. For example, specifying two levels of hierarchy requires a nonterminal production rule in addition to a terminal production rule, for example,

$$x_1 \rightarrow x_2x_2 \quad (\text{nonterminal production}) \quad \text{and} \quad (1)$$

$$x_2 \rightarrow AA \quad (\text{terminal production rule}). \quad (2)$$

In the example above, the x_2 nonterminal enables an intermediate level of hierarchy. A nonterminal can produce any finite-length permutation of higher-numbered nonterminals (for example, $x_1 \rightarrow x_2x_3$ is permitted but not $x_2 \rightarrow x_1x_3$ or $x_2 \rightarrow x_2x_3$). A nonterminal can also produce any finite-length permutation of terminals (for example, $x_1 \rightarrow AB$) or any finite-length permutation of mixed terminals and higher-numbered nonterminals (for example, $x_1 \rightarrow x_2B$ or $x_1 \rightarrow Bx_2$).

A complete syntax tree is built by fully expanding the nonterminal x_1 into a set of terminals. The production rules define the connectivity structure of the tree, where the parent node corresponds to the left side of the production rule and the child node(s) corresponds to the right side of the production rule. Across the entire syntax tree, each internal node corresponds to a nonterminal symbol and each leaf corresponds to a terminal symbol. Example syntax trees are provided in the main text figures.

A.1.2. CONSTRAINTS

A program in our language also requires a set of constraints, where a single constraint is defined with respect to a single node and all of its descendants in the syntax tree. Note that this includes constraints on the leaves of the tree (corresponding to the terminal symbols). More specifically, a constraint is a function that takes as input the (sub)tree, as

well as its corresponding (sub)sequence and (sub)structure, and outputs a real number. For example, a constraint defined with respect to the node corresponding to x_1 simply receives as input the entire syntax tree, the full-length sequence, and the full protein structure. The same constraint (i.e., the same function) can be applied to multiple nodes in the tree. We will use $f_j(x_i)$ to denote a constraint j defined with respect to the x_i node.

A.1.3. COMPILATION OF CONSTRAINTS INTO AN ENERGY FUNCTION

We compile a program into an energy function. In our study, we simply compute a linear combination of all the constraints in the user-specified set, i.e., $E(x) = \sum_i \sum_j f_j(x_i)$, where $f_j(x_i)$ is defined as zero when a constraint is not applied to a given node. In practice, we explicitly keep track of a scalar multiplicative weight on each constraint, i.e., $E(x) = \sum_i \sum_j a_j f_j(x_i)$. This energy is used in the simulated annealing optimization procedure described below. Specific examples of constraint functions used in our study are also provided below.

Linear combinations work well for our choice of generative model, but in principle any combination of the energy terms could be used here. For example, if we compiled our program into an energy function for a generative model that used a reward function (like a reinforcement learning agent), we might prefer a multiplicative combination of the inverse of our current energy terms.

A.1.4. SIMULATED ANNEALING

The energy function is used as part of an iterative black-box optimization loop, where over multiple iterations, a change to a given state (in this case, a protein sequence design) is accepted with some probability. We use a simulated annealing algorithm in which the acceptance probability is controlled by a temperature value such that the optimization can tolerate higher energy changes at the beginning of the optimization before favoring changes that decrease the energy toward the end of the optimization. In our study, we begin by initializing the sequence state (one unique sequence per terminal symbol) with uniform amino-acid probability to a given user-specified length; we also compute an initial structure prediction from this sequence.

Each iteration proposes a mutation to the protein sequence. To make this proposal, first, one of the terminal symbols is chosen with uniform probability, and second, one of a substitution, insertion, or deletion is chosen with some probability (we default to 60%, 20%, and 20%, respectively). For substitutions and insertions, the new amino acid is chosen with some probability (unless otherwise specified, we apply uniform probability over a reduced amino acid alphabet that excludes cysteine). We default to uniform probability over

用于生成蛋白质设计的高级编程语言

A. 方法

A.1. 高级编程语言和基于能量的优化

我们语言中的一个程序由（1）一棵语法树和（2）一组约束完全指定。该程序编译为一个能量函数，该函数用于指导蛋白质序列的黑箱优化，同时利用其预测结构。

A.1.1. 语法树

语法树由非终结符号组成，我们用 x_i 表示，以及终端符号，我们在示例中用大写字母字符表示，如 A, B, C 等。每个终端符号定义一个独特的蛋白质序列。非终结符号 x_1 被指定为特殊的起始符号；所有程序必须具有 x_1 。额外的非终结符号用于定义层次复杂性。例如，指定两个层次的层次结构需要一个非终结产生式规则，以及一个终端产生式规则，例如，

$$x_1 \rightarrow x_2x_2 \quad (\text{nonterminal production}) \quad \text{and} \quad (1)$$

$$x_2 \rightarrow AA \quad (\text{terminal production rule}). \quad (2)$$

在上述示例中， x_2 非终结符实现了一个中间层次的层次结构。一个非终结符可以产生任何有限长度的高编号非终结符的排列（例如， $x_1 \rightarrow x_2x_3$ 是允许的，但 $x_2 \rightarrow x_1x_3$ 或 $x_2 \rightarrow x_2x_3$ 不被允许）。一个非终结符也可以产生任何有限长度的终端的排列（例如， $x_1 \rightarrow AB$ ）或任何有限长度的混合终端与高编号非终结符的排列（例如， $x_1 \rightarrow x_2B$ 或 $x_1 \rightarrow Bx_2$ ）。

完整的语法树是通过充分展开非终端 x_1 为一组终端所构建的。产生式规则定义了树的连接结构，其中父节点对应于产生式规则的左侧，而子节点则对应于产生式规则的右侧。在整个语法树中，每个内部节点对应于一个非终端符号，每个叶子则对应于一个终端符号。主文本图中提供了示例语法树。

A.1. 2. 约束 在我们的语言中，程序还需要一组约束，其中单个约束相对于单个节点及其所有后代在语法树中定义。注意，这包括对树的叶子节点的约束（对应于终端符号）。更具体而言，约束是一个函数，它以（子）树作为输入，如下所示：

以及其对应的（子）序列和（子）结构，并输出一个实数。例如，与对应于 x_1 的节点相关的约束简单地接收整个语法树、全长序列和完整的蛋白质结构作为输入。同一约束（即相同的函数）可以应用于树中的多个节点。我们将使用 $f_j(x_i)$ 来表示相对于 x_i 节点定义的约束 j 。

A.1. 3. 将约束编译为能量函数 我们将程序编译为能量函数。在我们的研究中，我们简单地计算用户指定的集合中所有约束的线性组合，即 $E(x) = \sum_i \sum_j f_j(x_i)$ ，其中 $f_j(x_i)$ 在未应用于给定节点时定义为零。在实践中，我们明确跟踪每个约束的标量乘法权重，即 $E(x) = \sum_i \sum_j a_j f_j(x_i)$ 。该能量用于下面描述的模拟退火优化程序。我们研究中使用的约束函数的具体示例也在下面提供。线性组合在我们选择的生成模型中表现良好，但原则上可以在此使用能量项的任何组合。例如，如果我们将程序编译为一个使用奖励函数的生成模型的能量函数（如强化学习代理），我们可能更倾向于当前能量项的倒数的乘法组合。

A.1. 4. 模拟退火 能量函数作为迭代黑箱优化循环的一部分，其中在多次迭代中，对给定状态（在这种情况下为蛋白质序列设计）的变化以某种概率被接受。我们使用模拟退火算法，其中接受概率由温度值控制，从而使优化在开始时能够容忍更高能量变化，而在优化结束时更倾向于减少能量的变化。在我们的研究中，我们首先以均匀氨基酸概率初始化序列状态（每个终端符号一个唯一序列）至用户指定的长度；我们还从该序列计算初始结构预测。每次迭代为蛋白质序列提议突变。为了进行该提议，首先，均匀概率选择一个终端符号；其次，选择替换、插入或删除中的一个以一定概率（我们默认分别为60%、20%和20%）。对于替换和插入，新氨基酸是以一定概率选择的（除非另行指定，我们在简化氨基酸字母表上应用均匀概率，该字母表排除了半胱氨酸）。我们默认使用均匀概率

A high-level programming language for generative protein design

all possible sequence positions.

The next step in the iteration is to obtain a structure prediction corresponding to the sequence with the proposed change. This prediction provides the structural information that is used to compute the values of the individual constraint functions. These values are then combined to produce the value of the energy function, as described above. This energy function is evaluated on the overall design with and without the proposed mutation, which we denote $E(x^*)$ and $E(x^{(i)})$, respectively. The mutation is accepted with probability

$$\min \left(\exp \left\{ \frac{1}{T_i} E(x^*) - E(x^{(i)}) \right\}, 1 \right)$$

where T_i is the temperature parameter at iteration i that decays geometrically over the optimization. By default, our optimization leverages user-specified values $T_{\max}$ and $T_{\min}$, with a decay schedule given as

$$T_i = \left(\frac{T_{\min}}{T_{\max}} \right)^{i/M}$$

where M is the user-specified number of annealing steps. We report specific values for $T_{\max}$ and M in the experiment descriptions below and default to $T_{\min} = 0.0001$.

A.1.5. SINGLE CHAIN CONSTRAINT

Our language accommodates both single- and multi-chain design through the use of a special “single chain” constraint. By default, without this constraint applied, all terminal symbols are assumed to correspond to separate chains. When this constraint is applied to a given node, it constrains all of the terminal symbols to be part of a single chain according to the left-to-right order defined in the syntax tree. For example, consider a syntax subtree with $x_2 \rightarrow x_3AB$ and $x_3 \rightarrow CD$ productions. A single chain constraint applied to node x_2 would create a chain consisting of a contiguous sequence $CDAB$. Unlike other constraints, this constraint is enforced as part of structure prediction, prior to the energy function compilation.

A.2. Constraint implementation

A.2.1. ESMFOLD STRUCTURE PREDICTION

We obtain all-atom structure predictions using ESMFold (16), where the prediction is made over the entire protein sequence and is represented as a set of atomic coordinates and their corresponding residue identities and indices. This predicted structure is the basis for the structural information passed to each of the specific constraint functions. When a constraint is defined on a subtree, that constraint only has access to the structural information (atomic coordinates, etc.) of the sequence encoded by that subtree.

A.2.2. STRUCTURE PREDICTION CONFIDENCE (PTM AND PLDDT)

ESMFold produces a pTM score, which indicates the model’s confidence in the overall structure prediction, and a per-atom pLDDT score, which indicates the model’s confidence in the specific atomic coordinate prediction. The pTM value and the mean of the backbone pLDDT values are constraints that are meant to steer the optimization toward structures with higher structure prediction confidence, which is associated with naturally plausible and designable structures. We use a linear combination of the quantities $1 - \text{pTM}$ and $1 - \text{pLDDT}$ (since a higher confidence/lower energy is desirable), with user-specified weights, as the returned value of the confidence constraint.

A.2.3. SURFACE-EXPOSED HYDROPHOBICS

The surface exposed hydrophobics constraint aims to reduce the hydrophobicity of the protein surface, where high hydrophobicity leads to protein aggregation and insolubility. We implement this constraint using the Shrake-Rupley “rolling probe” algorithm to determine the surface exposed atoms (30) as implemented in the biotite Python package version 0.35.0 (31). We then calculate the fraction of atoms involved in hydrophobic residues that are also surface exposed, and we use this fraction as the output of the constraint function.

A.2.4. GLOBULARITY

It is sometimes desirable to encourage a protein chain to pack into a globular structure. Our globularity constraint is implemented by computing the centroid of a set of atomic coordinates, where the globularity constraint function returns the variance of the distances from all coordinates to this centroid. Intuitively, low variance indicates that all coordinates that are largely equidistant to the centroid, which is more consistent with globular packing.

A.2.5. SECONDARY STRUCTURE

The secondary structure constraint steers the energy toward user-defined secondary structure. To annotate residue secondary structure, we use the P-SEA algorithm (32) as implemented by the biotite Python package (31). This constraint function returns one minus the fraction of residues that belong to the desired secondary structure element (since a higher fraction/lower energy is desirable).

A.2.6. ROTATIONAL SYMMETRY

To design symmetry, we first find it helpful to tie the sequence identities across the subsequences corresponding to the asymmetric units. The first symmetry we consider is rotational symmetry, which only consider the centroids

一种用于生成蛋白质设计的高级编程语言

所有可能的序列位置。

迭代的下一步是获取与提议的变化对应的结构预测。该预测提供了用于计算各个约束函数值的结构信息。这些值随后被组合以生成能量函数的值，如上所述。该能量函数在整体设计中评估，考虑了提议的突变与不考虑突变的情况，我们用 $E(x)$ 表示。

$x^*)$ 和 $E(x^{(i)})$ ，分别。突变的接受概率为

$$\min \left(\exp \left\{ \frac{1}{T_i} E(x^*) - E(x^{(i)}) \right\}, 1 \right)$$

其中 T_i 是第 i 次迭代的温度参数，该参数在优化过程中以几何方式衰减。默认情况下，我们的优化利用用户指定的值 $T_{\max}$ 和 $T_{\min}$ ，衰减计划如下所示：

$$T_i = \left(\frac{T_{\min}}{T_{\max}} \right)^{i/M}$$

其中 M 是用户指定的退火步骤数。我们将在下面的实验描述中报告 $T_{\max}$ 和 M 的具体值，并默认为 $T_{\min} = 0.0001$ 。

A.1. 5. 单链约束 我们的语言通过使用特殊的“单链”约束，适应单链和多链设计。默认情况下，在未应用此约束的情况下，所有终端符号被假定为对应于独立的链。当此约束应用于给定节点时，它约束所有终端符号按照语法树中定义的从左到右的顺序成为单链的一部分。例如，考虑一个具有 $x_2 \rightarrow x_3AB$ 和 $x_3 \rightarrow CD$ 生成的语法子树。应用单链约束于节点 x_2 将创建一个由连续序列 $CDAB$ 组成的链。与其他约束不同，此约束作为结构预测的一部分执行，在能量函数编译之前。

A.2. 约束实现

A.2.1. ESMFold 结构预测

我们使用ESMFold获得全原子结构预测。(16)，其中预测是在整个蛋白质序列上进行的，并表示为一组原子坐标及其相应的残基身份和索引。该预测结构是传递给每个特定约束函数的结构信息的基础。当在子树上定义约束时，该约束仅能访问由该子树编码的序列的结构信息（原子坐标等）。

A.2.2. 结构预测置信度 (pTM 和 pLDDT)

ESMFold生成一个pTM评分，该评分指示模型对整体结构预测的信心，以及一个每个原子的pLDDT评分，该评分指示模型对特定原子坐标预测的信心。pTM值和主链pLDDT值的均值是旨在引导优化朝向具有更高结构预测信心的结构的约束，这与自然上可行和可设计的结构相关。我们使用数量的线性组合 $1 - \text{pTM}$ 和 $1 - \text{pLDDT}$ （由于更高的信心/更低的能量是可取的），并根据用户指定的权重，作为信心约束的返回值。

A.2. 3.S表面-暴露的疏水性暴露在表面的疏水性约束旨在减少蛋白质表面的疏水性，其中高疏水性会导致蛋白质聚集和不溶性。我们使用Shrake-Rupley “滚动探针”算法来确定暴露表面的原子 (30)，该算法在 biotite Python包版本0.35.0中实现 (31)。然后，我们计算参与疏水残基的原子中暴露在表面的原子所占的比例，使用该比例作为约束函数的输出。

A.2. 4. 球状性 有时希望鼓励蛋白质链打包成球状结构。我们的球状性约束是通过计算一组原子坐标的质心来实现的，其中球状性约束函数返回所有坐标到该质心的距离的方差。直观地说，低方差表明所有坐标与质心大致等距，这与球状打包更为一致。

A.2. 5. SECONDARYSTRUCTURE 二级结构约束将能量引导向用户定义的二级结构。为了注释残基的二级结构，我们使用P-SEA算法 (32)，该算法由biotite Python包 (31) 实现。该约束函数返回一个减去属于所需二级结构元素的残基比例的值（因为更高的比例/更低的能量是可取的）。

A.2. 6. 旋转对称性 为了设计对称性，我们首先发现有必要将与非对称单元相对应的子序列中的序列标识关联起来。我们考虑的对称性是旋转对称性，它仅考虑质心。

A high-level programming language for generative protein design

of the immediate children of the constraint’s node; for example, rotational symmetry defined on a node x_1 where $x_1 \rightarrow x_2x_3x_4$ would only consider the centroids of the individual substructures defined by x_2, x_3 , or x_4 .

Using the left-to-right order of these children in the production rule, the rotational symmetry function first computes the distances among adjacent centroids, circularly wrapping to include the distance between the first and last symbols; for example, rotational symmetry defined on $x_1 \rightarrow x_2x_3x_4$ would compute the set of distances among pairs (x_2, x_3) , (x_3, x_4) , and (x_4, x_2) . The final value returned by this constraint function is the variance among all adjacent distances; intuitively, a rotational or ring-like symmetry would have equal distances among centroids. This rotational symmetry function is adopted from that used by Wicky et al. (22).

A.2.7. GLOBULAR SYMMETRY

The globular symmetry constraint is defined on the centroids of the immediate children of the constraint’s node; for example, globular symmetry defined on a node x_1 where $x_1 \rightarrow x_2x_3x_4$ would only consider the centroids of the individual substructures defined by x_2, x_3 , or x_4 (this is similar to rotational symmetry described above). The globularity symmetry function computes all pairwise distances among centroids and returns the variance of these distances. In practice, this constraint function is useful for defining symmetry that is not rotational, for example, the symmetry observed in polyhedral assemblies.

A.2.8. ALL-ATOM COORDINATION

One approach to designing functional proteins is to constrain (a portion of) the protein to match the structure of a known functional site in nature. We accomplish this with an all-atom coordination constraint. This constraint is first defined with respect to a list of atoms from a native protein structure (outside of our designed protein), which we denote y_{native} . We then constrain all of the atoms in the corresponding (sub)tree to match, which we denote y_{design} , as closely as possible, the coordination of the atoms in y_{native} . We achieve this with two functions. The first is the constrained root mean square deviation (cRMSD),

$$\text{cRMSD}(y_{\text{native}}, y_{\text{design}}) = \min_T \left(\frac{1}{n} \sum_{i=1}^n \|a_i(y_{\text{native}}) - T(a_i(y_{\text{design}}))\|_2^2 \right)^{1/2} \quad (3)$$

where T is a structural transformation, a_i denotes the atomic coordinates of the i th atom out of n atoms considered, and $\|\cdot\|$ denotes a vector norm. We implement the structural alignment using the Kabsch algorithm (33) as implemented by biotite (31). The second function for constraining atomic

coordination is the distance-matrix RMSD (dRMSD),

$$\text{dRMSD}(y_{\text{native}}, y_{\text{design}}) = \left(\frac{2}{n(n-1)} \sum_{i=1}^n \sum_{j=1}^n (d_{ij}(y_{\text{native}}) - d_{ij}(y_{\text{design}}))^2 \right)^{1/2} \quad (4)$$

where d_{ij} is the Euclidean distance between the i th and j th atoms. The returned final value is a linear combination of the cRMSD and dRMSD values with user-specified weights. In practice, cRMSD is sometimes excluded (i.e., its weight is set to zero) in conjunction with dRMSD, as cRMSD alone does not appear sufficiently stable to create a sufficiently smooth energy landscape.

A.2.9. BACKBONE ATOM COORDINATION

For a class of design tasks called fixed backbone design, we desire to only constrain the backbone atoms of the protein structure and have the optimization produce sequences that match a known backbone. This constraint is largely equivalent to the all-atom constraint described above, but rather than constraining all atoms (including side chains), this constraint is only applied to the carbon, α -carbon, and nitrogen atoms in the protein backbone.

A.2.10. SURFACE EXPOSURE

In some cases, we desire that a given set of residues be exposed on the surface of the protein (for example, when scaffolding a protein binding site). As with the hydrophobics constraint, we leverage the Shrake-Rupley algorithm (30) as implemented by biotite (31). We then calculate the fraction of surface exposed atoms within the structure corresponding to the constraint’s subtree, and we use one minus this fraction as the output of the function.

A.2.11. LENGTH

The length constraint requires a user-specified number of residues. In practice, we can enforce a hard length constraint by disallowing insertions and deletions during the optimization procedure, or through a function that returns increasingly high values when a sequence length goes beyond a user-specified range. In this study, whenever we apply a length constraint we take the former approach.

A.3. Design tasks and experiments

A.3.1. FREE HALLUCINATION

Free hallucination simply requires applying confidence and surface-exposed hydrophobic constraints to the whole protein, where we place equal weight on each term (pTM, pLDDT, and hydrophobics). In the experiments described in this study, we ran simulated annealing over 30,000 iter-

用于生成蛋白质设计的高级编程语言

约束节点的直接子节点; 例如, 定义在节点上的旋转对称性 x_1 , 仅会考虑由 x_2 、 x_3 或 x_4 定义的个别子结构的重心。

根据这些子元素在生成规则中的从左到右的顺序, 旋转对称性函数首先计算相邻质心之间的距离, 并通过循环连接来包括第一个和最后一个符号之间的距离; 例如, 在 $x_1 \rightarrow x_2x_3x_4$ 上定义的旋转对称性将计算一组距离, 其中的对包括 (x_2, x_3) 、 (x_3, x_4) 和 (x_4, x_2) 。该约束函数最终返回的值是所有相邻距离之间的方差; 直观地讲, 旋转或环状对称性在质心之间会有相等的距离。这个旋转对称性函数来源于Wicky等人使用的函数 (22)。

A.2. 7. 球形对称 球形对称约束定义在约束节点的直接子节点的质心上; 例如, 定义在一个节点 x_1 的球形对称 $x_1 \rightarrow x_2x_3x_4$ 仅考虑由 x_2 、 x_3 或 x_4 定义各个子结构的质心 (这与上述描述的旋转对称性相似)。球形对称函数计算所有质心之间的成对距离, 并返回这些距离的方差。在实践中, 这个约束函数对于定义非旋转对称性是有用的, 例如在多面体组合中观察到的对称性。

A.2. 8. 全原子协调 设计功能性蛋白的一种方法是将蛋白质的 (部分) 结构约束为与自然界已知功能位点的结构相匹配。我们通过全原子协调约束实现这一点。该约束首先是相对于来自本地蛋白质结构 (在我们设计的蛋白质之外) 的原子列表定义的, 我们称之为 y_{native} 。然后, 我们约束相应 (子) 树中的所有原子, 以尽可能接近地匹配 y_{native} 中原子的协调, 我们将其称为 y_{design} 。我们通过两个函数来实现这一点。第一个是受约束的均方根偏差 (cRMSD),

$$\text{cRMSD}(y_{\text{native}}, y_{\text{design}}) = \min_T \left(\frac{1}{n} \sum_{i=1}^n \|a_i(y_{\text{native}}) - T(a_i(y_{\text{design}}))\|_2^2 \right)^{1/2} \quad (3)$$

其中 T 是一种结构变换, a_i 表示考虑的 n 个总原子中的 i 号原子的原子坐标, 并且 $\|\cdot\|$

表示一个向量范数。我们使用Kabsch算法 (33) 实现结构对齐, 该算法由biotite实现 (31)。约束原子的位置的第二个函数

协同是距离矩阵RMSD (dRMSD),

$$\text{dRMSD}(y_{\text{native}}, y_{\text{design}}) = \left(\frac{2}{n(n-1)} \sum_{i=1}^n \sum_{j=1}^n (d_{ij}(y_{\text{native}}) - d_{ij}(y_{\text{design}}))^2 \right)^{1/2} \quad (4)$$

其中 d_{ij} 是第 i 个和第 j 个原子之间的欧几里得距离。返回的最终值是 cRMSD 和 dRMSD 值的线性组合, 权重由用户指定。在实践中, cRMSD 有时被排除在外 (即其权重被设为零), 与 dRMSD 结合使用, 因为仅仅依靠 cRMSD 似乎并不足够稳定, 以创建一个足够平滑的能量景观。

A.2. 9. 骨架原子协调 对于称为固定骨架设计的一类设计任务, 我们希望仅对蛋白质结构的骨架原子施加约束, 并使优化产生与已知骨架匹配的序列。此约束在很大程度上等同于上述描述的全原子约束, 但该约束仅适用于蛋白质骨架中的碳、 α -碳和氮原子, 而不是限制所有原子 (包括侧链)。

A.2. 10. 表面暴露 在某些情况下, 我们希望特定的残基集在蛋白质的表面上暴露 (例如, 构建蛋白质结合位点时)。与疏水约束一样, 我们利用 Shrake-Rupley 算法

(30) 由 biotite 实现。我们计算对应于约束的子树结构中表面暴露原子的比例, 并使用 1 减去该比例作为函数的输出。

A.2.11. 长度

长度约束要求用户指定数量的残基。在实践中, 我们可以通过在优化过程中禁用插入和删除来强制实施严格的长度约束, 或者通过一个函数, 在序列长度超出用户指定范围时返回逐渐增高的值。在本研究中, 每当我们应用长度约束时, 我们采取前者的方法。

A.3. 设计任务与实验

A.3.1. 自由幻觉

自由幻觉仅仅需要对整个蛋白质施加信心和表面暴露的疏水性约束, 其中我们对每个项 (pTM、pLDDT 和疏水性) 赋予相等的权重。在本研究中所描述的实验中, 我们进行了30,000次迭代的模拟退火。

A high-level programming language for generative protein design

ations with $T_{\max} = 1$ across 200 seeds. We also evaluated single-sequence AlphaFold2 (18) on the final sequences produced by these 200 optimization runs.

A.3.2. FIXED BACKBONE DESIGN

For fixed backbone design, we apply a weight of 2 on the dRMSD constraint, a weight of 1 on the cRMSD, pTM, and pLDDT constraints, and a weight of 0.5 on the hydrophobics constraint. As the target backbones, we used the de novo structures with PDB IDs 1QYS, 5L33, 6D0T, 6MRS, 6W3W, and 6WVS. In the experiments described in this study, we ran simulated annealing over 30,000 iterations with $T_{\max} = 1$ across at least 50 seeds for each de novo backbone.

A.3.3. SECONDARY STRUCTURE DESIGN

We performed protein design with partial constraints on a protein by constraining the secondary structure corresponding to different segments of the protein sequence. We place a weight of 10 on the secondary structure constraint and weights of 1 on pTM, pLDDT and hydrophobics constraints. Our programs specify the secondary structure corresponding to two discrete subsequences, where we program (1) all alpha, (2) all beta, and (3) mixed alpha and beta secondary structure. We ran simulated annealing over 30,000 iterations $T_{\max} = 1$ for 10 seeds for each of these three programs (30 optimization trajectories in total).

A.3.4. SINGLE FUNCTIONAL SITE SCAFFOLDING

To program functional site scaffolding on a de novo backbone, we divide a single-chain sequence into three segments: a sequence in the middle segment (with an all-atom coordination constraint and a surface exposure constraint) flanked by two “free” sequences. pTM, pLDDT, and hydrophobics constraints are also applied to the full protein. We apply a weight of 2 to the cRMSD and dRMSD constraints, and a weight of 1 to the pTM, pLDDT, and hydrophobics constraints.

We attempted to scaffold five protein binding sites, the first three of which were successfully scaffolded by Wang et al. (20):

1. IL10: We used the residue indices 31–40, inclusive, of chain L in the PDB structure 1Y6K, corresponding to the IL10 binding site of IL-10R1 (34).
2. ACE2: We used the residue indices 5–23, inclusive, of chain A in the PDB structure 6M0J, corresponding to the ACE2 binding site of the SARS-CoV-2 spike receptor binding domain (RBD) (35).
3. C3d: We used the residue indices 104–126 and 170–184, inclusive, of chain A in the PDB structure 1GHQ,

corresponding to the C3d binding site of complement receptor 2 (36).

4. HA2: We used the residue indices 14–21, 33–42, and 45–49, inclusive, of chain B in the PDB structure 5JW3, corresponding to the influenza HA2 epitope of the antibody MEDI8825 (37).
5. RBD: We used the residue indices 439–450 and 498–506, inclusive, of chain C in the PDB structure 7MMO, corresponding to the SARS-CoV-2 RBD epitope of the antibody bebtelovimab (38).

We ran simulated annealing over 30,000 iterations with $T_{\max} = 1$ for 1,000 seeds for each of the five binding sites (5,000 optimization trajectories in total).

A.3.5. SYMMETRIC AND HOMO-OLIGOMER DESIGN

We first program single-chain symmetry using a rotational symmetry constraint applied to the top-level node. In our program, we also tie the sequences across the subsequences corresponding to the asymmetric units such that we only use a single terminal symbol; an example program for designing 3-fold symmetry is provided in Figure 3A. We place a weight of 1 on the symmetry constraint, as well as weights of 1 on pTM, pLDDT, and hydrophobics constraints. We also place length constraints on the terminal nodes. We specify programs where we increase the fold-symmetry from 3- to 8-fold. We also vary the lengths that constrain the terminal symbol such that the full sequence has approximately 200, 300, or 400 residues (for example, a 200-residue protein with 3-fold symmetry would have length constraints of 66 on its terminal symbols). We ran simulated annealing over 30,000 iterations with a starting temperature of 1 for 10 seeds for each of the six fold symmetries and each of the three length constraints (for a total of 180 optimization trajectories).

We also designed larger homo-oligomers similarly, but removing the single-chain constraint from the top-level node. We designed trimeric, tetrameric, hexameric, and octameric homo-oligomers with a globular symmetry constraint applied to the top level node. We placed a weight of 1 on the symmetry constraint, as well as weights of 1 on pTM, pLDDT, and hydrophobics constraints. We also placed a weight of 0.1 on globularity constraints that are applied to each terminal symbol. We applied length constraints such that the full complex contained 720 residues (for example, the hexamer would consist of length-120 protomers). We ran simulated annealing over 30,000 iterations with $T_{\max} = 1$ for 10 seeds for each of oligomerization levels (for a total of 40 optimization trajectories).

用于生成蛋白质设计的高级编程语言

使用 $T_{\max} = 1$ 在200个种子上进行的计算。我们还对这200次优化运行所生成的最终序列进行了单序列 AlphaFold2 (18) 的评估。

A.3.2. 固定骨架设计

在固定骨架设计中，我们对 dRMSD 约束施加权重 2，对 cRMSD、pTM 和 pLDDT 约束施加权重 1，对疏水性约束施加权重 0.5。作为目标骨架，我们使用了具有 PDB ID 的 de novo 结构，分别为 1QYS、5L33、6D0T、6MRS、6W3W 和 6WVS。在本研究中描述的实验中，我们对每个 de novo 骨架进行了至少 50 个种子的 30,000 次迭代的模拟退火。

A.3. 3. 二级结构设计 我们通过约束与蛋白质序列不同片段对应的二级结构，执行了具有部分约束的蛋白质设计。我们对二级结构约束施加权重 10，对 pTM、pLDDT 和疏水性约束施加权重 1。我们的程序指定了对应于两个离散子序列的二级结构，其中我们编程 (1) 全部为 alpha, (2) 全部为 beta, 以及 (3) 混合 alpha 和 beta 的二级结构。我们对每个程序进行了 30,000 次迭代的模拟退火 $T_{\max} = 1$ ，为这三个程序中的每一个提供了 10 个种子（共 30 条优化轨迹）。

A.3. 4. 功能位点支架 为在de novo骨架上编程功能位点支架，我们将单链序列划分为三个部分：中间部分的序列（具有全原子协调约束和表面暴露约束），并由两个“自由”序列夹住。pTM、pLDDT和疏水性约束也应用于完整的蛋白质。我们对cRMSD和dRMSD约束施加权重2，对pTM、pLDDT和疏水性约束施加权重1。我们尝试支架五个蛋白质结合位点，其中前三个由Wang等成功支架。

(20):

1. IL10：我们使用PDB结构1Y6K中链L的残基索引31–40（含），对应于IL-10R1的IL10结合位点 (34)。
2. ACE2：我们使用了PDB结构6M0J中链A的残基索引5–23（包括），对应于SARS-CoV-2刺突受体结合域（RBD）的ACE2结合位点 (35)。
3. C3d：我们使用了PDB结构1GHQ中链A的残基索引104–126和170–184（包括在内），

对应于补体受体2的C3d结合位点 (36)。

4. HA2：我们使用了PDB结构5JW3中链B的残基索引14–21、33–42和45–49（包括），对应于抗体MEDI8825的流感HA2表位 (37)。
5. RBD：我们使用了PDB结构7MMO中链C的残基索引439–450和498–506（包括），对应于抗体 bebtelovimab 的SARS-CoV-2 RBD表位 (38)。

我们在30,000次迭代上运行了模拟退火， $T_{\max} = 1$ 每个结合位点使用1,000个种子（总共5,000条优化轨迹）。

A.3.5. 对称和同聚体设计

我们首先通过应用于顶级节点的旋转对称性约束来编程单链对称性。在我们的程序中，我们还将对应于不对称单元的子序列之间的序列连接，以便只使用单一终端符号；在图3A中提供了一个设计3倍对称性的示例程序。我们对对称性约束设置权重为1，以及对 pTM、pLDDT和疏水性约束也设置权重为1。我们还对终端节点设置了长度约束。我们指定了使我们将折叠对称性从3倍增加到8倍的程序。我们还改变了限制终端符号的长度，使得完整序列的残基数量大约为 200、300或400个（例如，具有3倍对称性的200个残基的蛋白质将对其终端符号设置66的长度约束）。我们在30,000次迭代中运行了模拟退火，初始温度为1，每种折叠对称性和每种长度约束都有10个种子（总共 180条优化轨迹）。

我们还以类似方式设计了更大的同源寡聚体，但从顶级节点中移除了单链约束。我们设计了三聚体、四聚体、六聚体和八聚体的同源寡聚体，并对顶级节点应用了球形对称约束。我们对对称约束赋予了1的权重，同时对pTM、pLDDT和疏水性约束也给予了1的权重。我们对每个终端符号应用的球状性约束赋予了0.1的权重。我们应用了长度约束，使得完整的复合物包含 720个残基（例如，六聚体由长度为120的前体组成）。我们在30,000次迭代上进行了模拟退火，与 $T_{\max} = 1$ 进行10种种子的每一层级的寡聚化（总共进行40条优化轨迹）。

A high-level programming language for generative protein design

A.3.6. TWO-LEVEL SYMMETRY DESIGN

We program two levels of symmetry using the productions

$$x_1 \rightarrow x_2 \cdots \quad (\text{top-level}) \quad \text{and} \quad (5)$$

$$x_2 \rightarrow A \cdots \quad (\text{bottom-level}). \quad (6)$$

In these programs, we place the single-chain constraint on x_2 , so the final designs are protein homo-oligomers. We place a globularity symmetry constraint on x_1 ; to control the top-level symmetry, we repeat x_2 according to the desired oligomerization. We place a rotational symmetry constraint on x_2 ; to control the bottom-level symmetry, we repeat A according to desired fold symmetry. We also place pTM, pLDDT, and hydrophobics constraints on the full protein; we place globularity constraints on x_2 . We compile constraints into an energy function with weights of 1 on all terms.

We enumerated programs over the grid varying both the top and bottom levels of symmetry from 2 to 4. We constrained lengths to 200 residues in total for the dimer of 2-fold; length-250 for the dimer of 3-fold; length-400 for the dimer of 4-fold, the trimer of 2-fold, the trimer of 3-fold, and the tetramer of 2-fold; length-450 for the trimer of 4-fold and the tetramer of 3-fold; and length-500 for the tetramer of 4-fold. We ran simulated annealing over 30,000 iterations with $T_{\max} = 1$ for 10 seeds for each of these programs (for a total of 90 optimization trajectories).

A.3.7. STRUCTURAL NOVELTY

We quantify a given design for structural novelty by running an exhaustive search over the PDB version 2022-08 (<http://www.rcsb.org/>) (39) to find the experimental structure with the highest TM-score to the designed structure, normalizing by the designed structure length, using TM-align version 20210107 (40).

A.3.8. INVERSE FOLDING ROUNDTRIP EXPERIMENTS

We assessed the “designability” of a structure prediction produced by our optimization procedure by “roundtripping” the protein through an inverse folding model. More specifically, given a predicted structure from our optimization loops, we first use ESM-IF1 (21), an independently trained inverse folding model, to sample 10 sequences with temperature 0.1 from the backbone coordinates. We then run these sequences through ESMFold and compute the cRMSD between the starting and the roundtripped backbone atoms of the predicted structure.

We performed this roundtrip experiment for 1,000 predicted structures that were obtained by first uniformly sampling one of the 180 symmetric single-chain optimization trajectories and then uniformly sampling one of the intermediate structure predictions within a given design loop (i.e., we do

not restrict this analysis to the best pLDDT structure over a design loop, which are highly biased toward high pLDDTs). We report the relationship between ESM-IF1 perplexities of all sample structures and the corresponding cRMSD values. We also report the relationship between the pLDDT of the starting structure and the minimum RMSD over the structure for the 10 inverse-folded sequences. We also repeated the same experiment for 1,000 predicted structures that were obtained by first uniformly sampling over the 90 two-level symmetry optimization trajectories and then uniformly sampling one of the intermediate structure predictions within a given design loop. We also report the same metrics as in the single-chain evaluation.

A.3.9. SYMMETRIC FUNCTIONAL SITE SCAFFOLDING

We designed proteins that symmetrically scaffold multiple functional sites by using the tree described for the single-site functional scaffold but replicating it according to the desired fold symmetry and adding a rotational symmetry constraint to the top-level node; an example program for a 3-fold functional site scaffold can be found in Figure 5A. We use weights of 10 on the cRMSD and dRMSD constraints and weights of 1 on the pTM, pLDDT, rotational symmetry, binding site surface exposure, and hydrophobics constraints. We ran simulated annealing over 30,000 iterations with a starting temperature of 1 over 20 seeds for the design of 3-fold scaffolds of the IL10 and ACE2 binding sites described above, as well as 20 seeds for the design of 5-fold scaffolds of the ACE2 binding site (for a total of 60 optimization loops).

A.3.10. HIERARCHICAL ASYMMETRIC SYMMETRY DESIGN

We increased the level of hierarchical complexity in our programs by designing with three levels of constraints. The top level specifies two asymmetric subunits. Each asymmetric subunit itself has two-level symmetry (similar to the setting described above): we specifically consider the dimer of 2-fold (2x2), the dimer of 3-fold (2x3), and the trimer of 2-fold (3x2). We write programs consisting of (1) two asymmetric 2x2s complexed together, (2) a 2x2 and a 3x2 complexed together, and (3) a 2x2 and a 2x3 completed together. An example program for the two asymmetric 2x2s is provided in Figure 5C. We use weights of 1 on all constraints (pTM, pLDDT, hydrophobics, rotational/globular symmetry, and globularity). We ran simulated annealing over 30,000 iterations with $T_{\max} = 1$ over 10 seeds for each of the three programs described above (for a total of 30 optimization loops).

生成蛋白质设计的高级编程语言

A.3. 6. 双层对称设计 我们使用这些生成物对两级对称进行编程

$$x_1 \rightarrow x_2 \cdots \quad (\text{top-level}) \quad \text{and} \quad (5)$$

$$x_2 \rightarrow A \cdots \quad (\text{bottom-level}). \quad (6)$$

在这些程序中, 我们将单链约束放置在 x_2 上, 因此最终设计为蛋白质同源寡聚体。我们将球状性对称性约束放置在 x_1 上; 为了控制顶级对称性, 我们根据所需的寡聚化重复 x_2 。我们将旋转对称性约束放置在 x_2 上; 为了控制底层对称性, 我们根据所需的折叠对称性重复 A 。我们还对整个蛋白质施加了pTM、pLDDT和疏水性约束; 我们对 x_2 施加了球状性约束。我们将约束编译成一个能量函数, 所有项的权重均为1。

我们在网格上列举了程序, 变化的对称性从2到4, 分别限制了长度: 对于2倍二聚体总计200个残基; 对于3倍二聚体250个残基; 对于4倍二聚体、2倍三聚体和3倍三聚体以及2倍四聚体400个残基; 对于4倍三聚体和3倍四聚体450个残基; 对于4倍四聚体500个残基。我们进行了30,000次迭代的模拟退火, 针对这些程序的每个种子运行了10次 (总共90条优化轨迹)。

A.3. 7. 结构新颖性我们通过对PDB版本2022-08进行全面搜索来量化给定设计的结构新颖性 (<http://www.rcsb.org/>) (39), 以找到与设计结构有最高TM-score的实验结构, 并通过设计结构的长度进行归一化, 使用TM-align版本20210107 (40)。

A.3. 8. I逆折叠往返实验我们通过将蛋白质传递通过逆折叠模型来评估我们优化程序生成的结构预测的“设计性”。更具体地说, 给定我们优化循环中的一个预测结构, 我们首先使用ESM-IF1 (21), 一个独立训练的逆折叠模型, 从主链坐标中以温度0.1采样10个序列。然后, 我们将这些序列通过ESMFold运行, 并计算预测结构的起始主链原子与往返主链原子之间的cRMSD。我们对1,000个预测结构进行了此次往返实验, 这些结构是通过首先均匀采样180个对称单链优化轨迹之一, 然后在给定设计循环内均匀采样一个中间结构预测获得的 (即, 我们执行

不将此分析限制于设计循环中的最佳pLDDT结构 (该结构高度偏向高pLDDT)。我们报告了所有样本结构的ESM-IF1困惑度与相应的cRMSD值之间的关系。我们还报告了起始结构的pLDDT与10个逆折叠序列的最小RMSD之间的关系。我们还对1,000个预测结构进行了相同的实验, 这些结构是通过首先在90条二级对称优化轨迹上均匀采样, 然后在给定设计循环内均匀采样其中一个中间结构预测得到的。我们还报告了与单链评估相同的指标。

A.3. 9. S对称功能位点支架我们设计了能够对称支撑多个功能位点的蛋白质, 采用了为单一功能支架描述的树结构, 但根据所需的折叠对称性进行复制, 并向顶层节点添加了旋转对称约束; 关于3倍功能位点支架的示例程序可以在图5A中找到。我们在cRMSD和dRMSD约束上使用了权重10, 且在pTM、pLDDT、旋转对称、结合位点表面暴露和疏水性约束上使用了权重1。我们在30,000次迭代中进行了模拟退火, 起始温度为1, 针对上述IL10和ACE2结合位点的3倍支架设计进行了20个种子的优化, 以及针对ACE2结合位点的5倍支架设计进行了20个种子的优化 (总共进行了60个优化循环)。

A.3. 10. H层次不对称对称设计我们通过设计三层约束, 提高了程序的层次复杂性。最上层指定了两个不对称单元。每个不对称单元本身具有两层对称性 (类似于上述设置): 我们特别考虑2倍 (2x2) 二聚体、3倍 (2x3) 二聚体和2倍 (3x2) 三聚体。我们编写的程序包括 (1) 两个不对称的2x2复合在一起, (2) 一个2x2和一个3x2复合在一起, 以及 (3) 一个2x2和一个2x3复合在一起。关于两个不对称的2x2的示例程序见图5C。我们对所有约束 (pTM、pLDDT、疏水性、旋转/球形对称和球状性) 使用权重1。我们在30,000次迭代上运行了模拟退火, 以对上述三个程序的每个程序进行10个种子 (总计30次优化循环)。

A high-level programming language for generative protein design

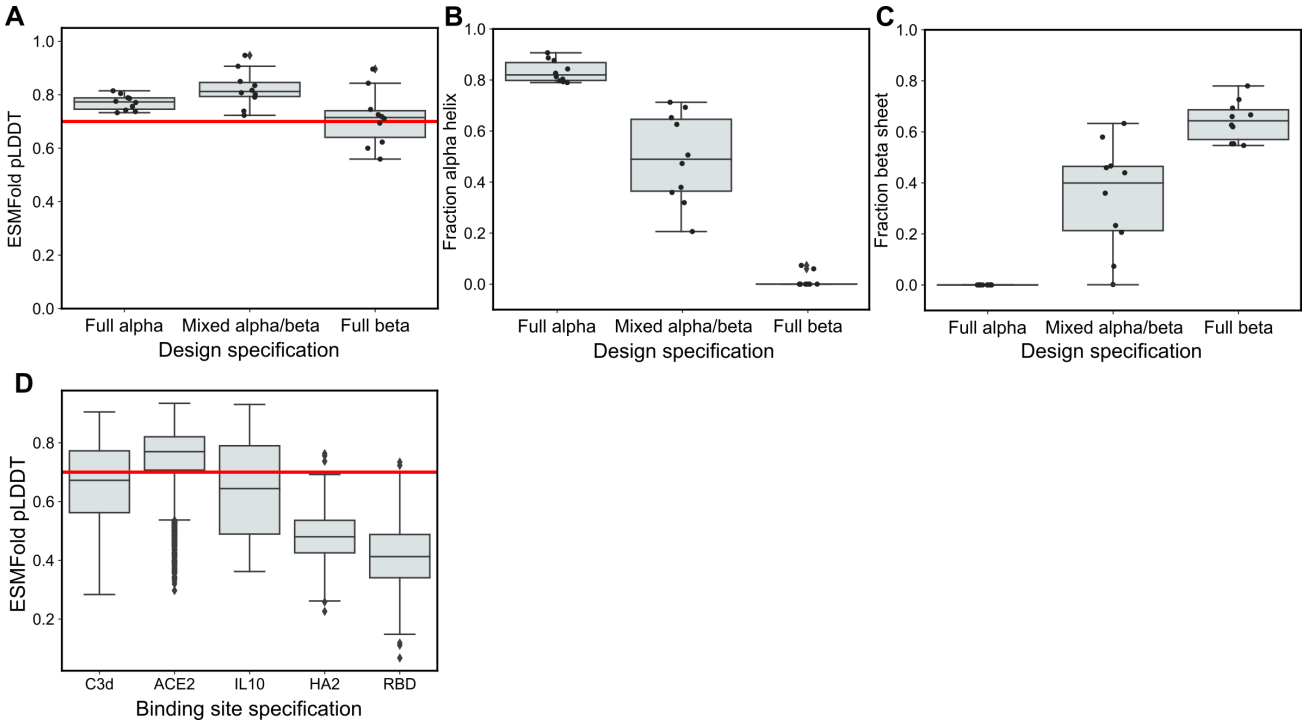


Figure S1. Additional plots for secondary structure design and functional site scaffolding. (A) ESMFold pLDDT values for different secondary structure design specifications (10 seeds per specification). A red line is plotted at pLDDT = 0.7. (B) The fraction of residues that are part of alpha helices for different secondary structure design specifications (10 seeds per specification). (C) The fraction of residues that are part of beta sheets for different secondary structure design specifications (10 seeds per specification). (D) ESMFold pLDDT values for different functional site scaffolding design runs (1,000 seeds per binding site).

用于生成蛋白质设计的高级编程语言

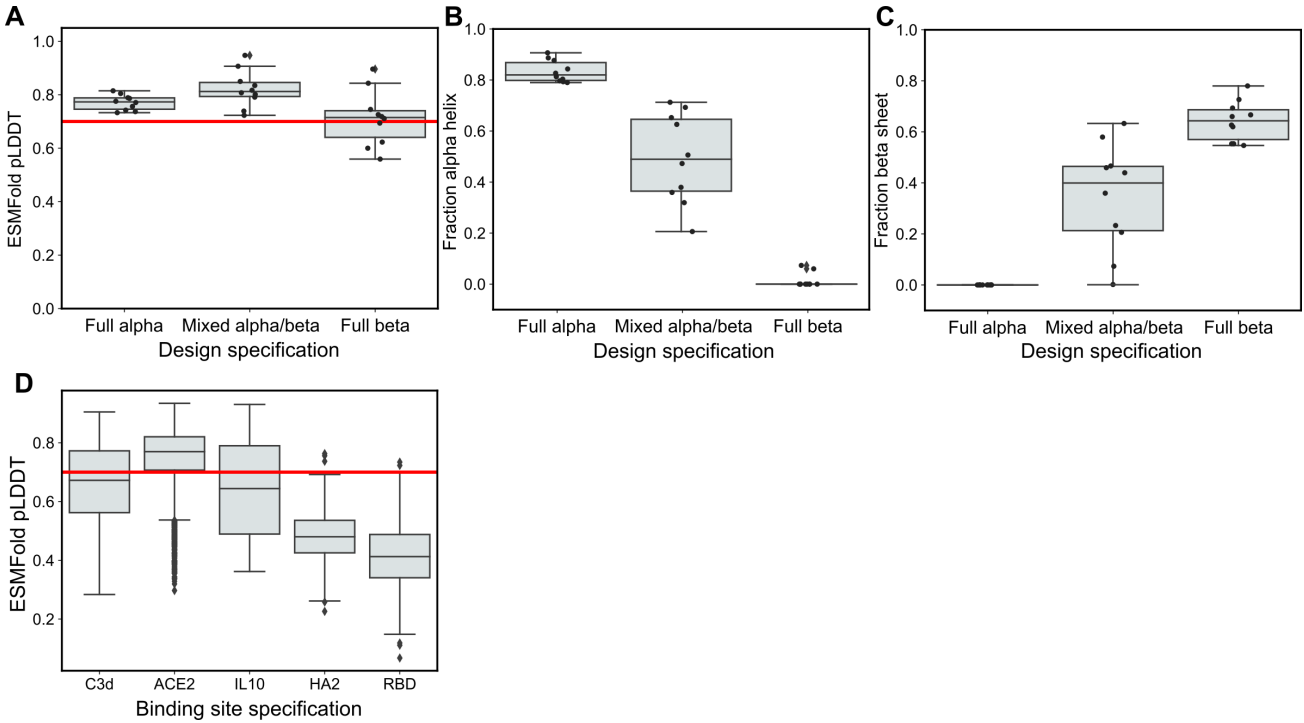


图 S1. 二级结构设计和功能位点支架的额外图形。(A) 针对不同二级结构设计规格的 ESMFold pLDDT 值 (每个规格 10 个种子)。在 pLDDT = 0.7 处绘制了一条红线。(B) 针对不同二级结构设计规格的属于 α 螺旋的残基比例 (每个规格 10 个种子)。(C) 针对不同二级结构设计规格的属于 β 薄片的残基比例 (每个规格 10 个种子)。(D) 针对不同功能位点支架设计运行的 ESMFold pLDDT 值 (每个结合位点 1,000 个种子)。

A high-level programming language for generative protein design

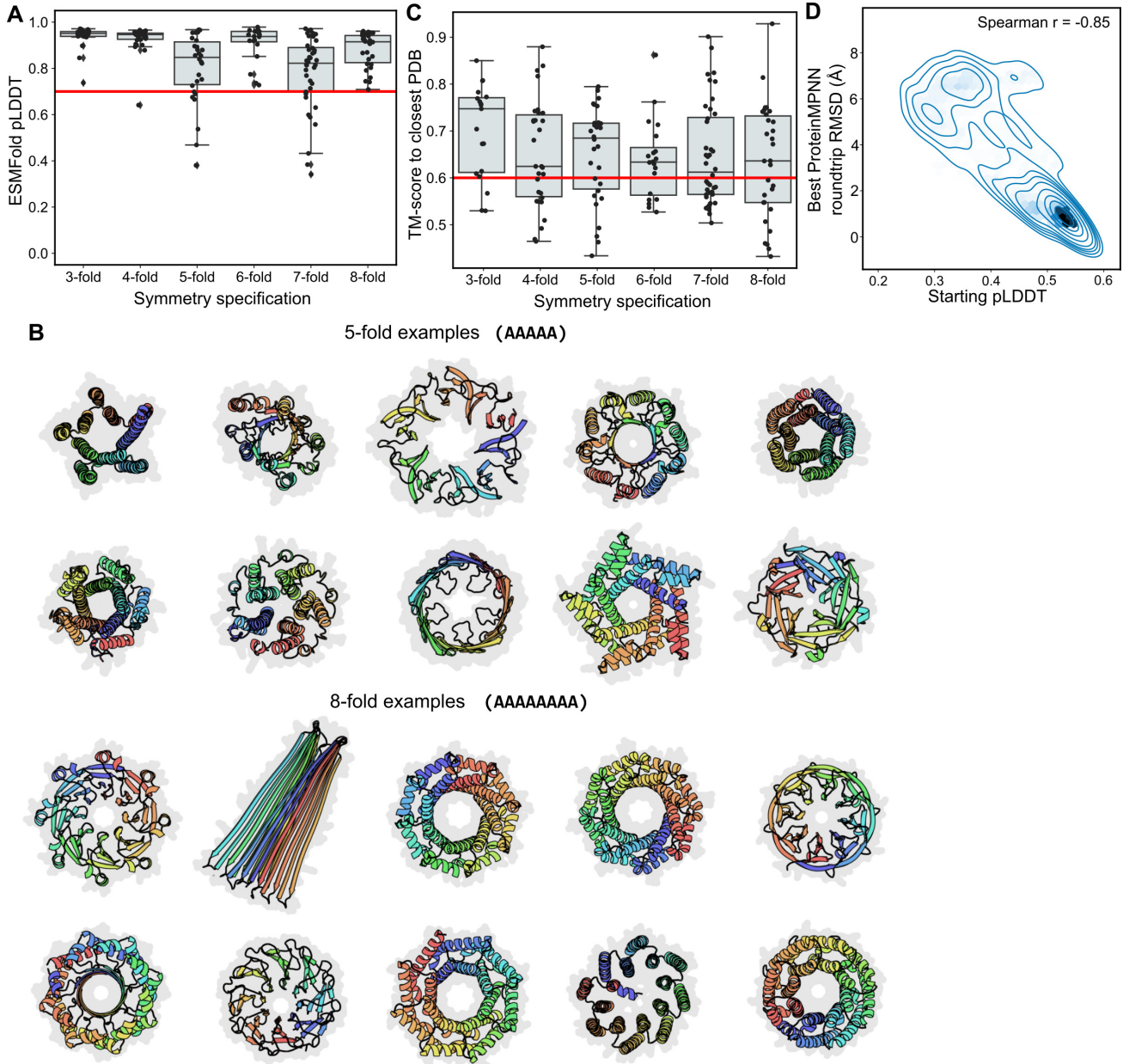


Figure S2. Additional plots for the design of symmetric single chains. (A) ESMFold pLDDT values for different fold symmetry design specifications (30 seeds per specification). A red line is plotted at pLDDT = 0.7. (B) Ten randomly sampled designs for the design of 5- and 8-fold symmetry. (C) TM-scores for different fold symmetry design specifications (30 seeds per specification); the TM-score is between the best design and the closest structure in the PDB. A red line is plotted at TM-score = 0.6. (D) Samples obtained by inverse folding with ProteinMPNN. On the x -axis is the pLDDT of the designed structure prior to the roundtrip and on the y -axis is the roundtrip RMSD.

用于生成蛋白质设计的高级编程语言

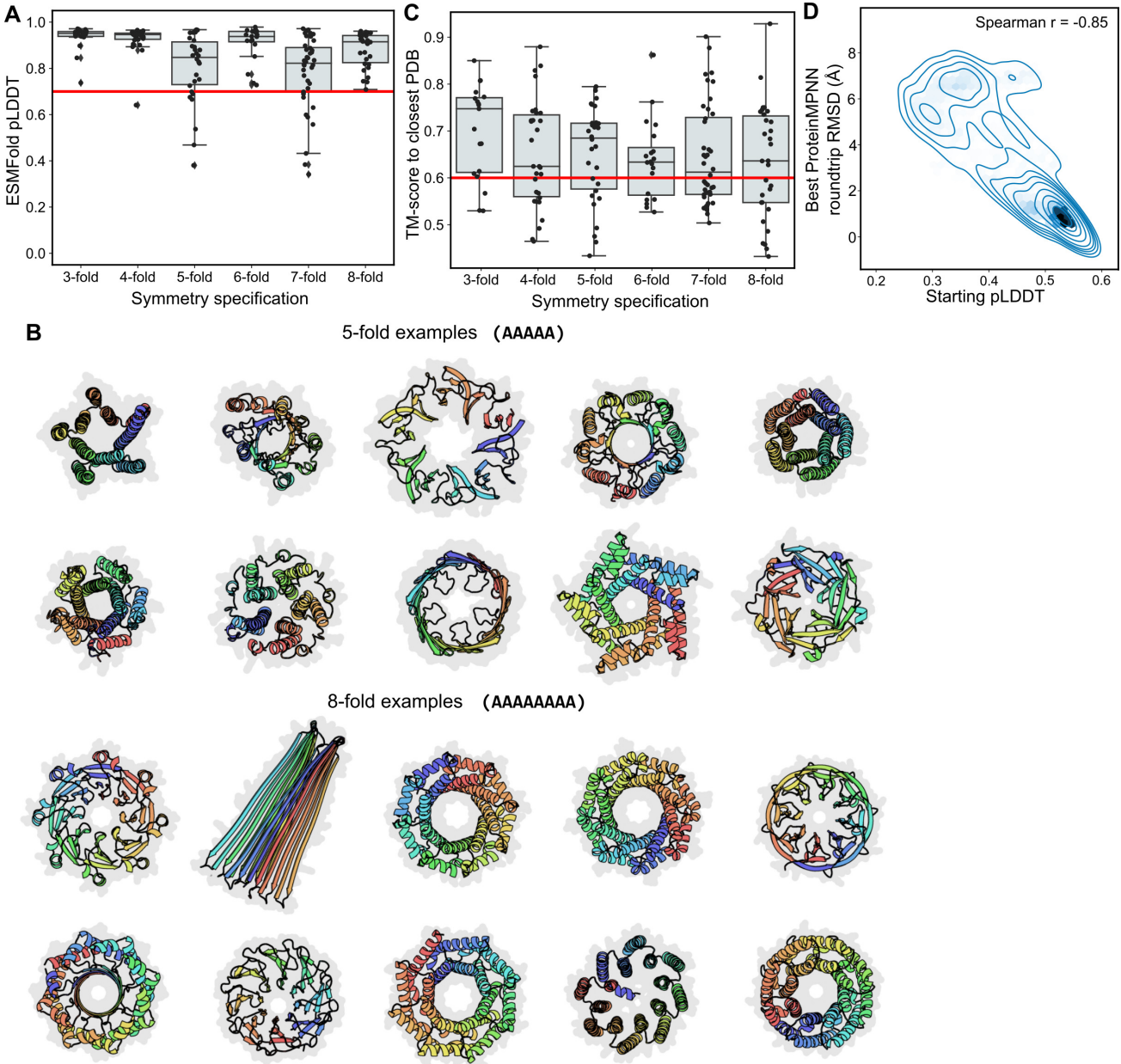


图 S2. 对称单链设计的附加图。(A) 对不同折叠对称性设计规范的 ESMFold pLDDT 值 (每个规范 30 个种子)。在 pLDDT = 0.7 处绘制一条红线。(B) 随机采样的十个 5 重和 8 重对称性设计。(C) 对不同折叠对称性设计规范的 TM 分数 (每个规范 30 个种子); TM 分数是在最佳设计和 PDB 中最接近的结构之间。在 TM 分数 = 0.6 处绘制一条红线。(D) 通过 ProteinMPNN 逆折叠获得的样本。在 x 轴上是设计结构在回程之前的 pLDDT, 而在 y 轴上是回程 RMSD。

A high-level programming language for generative protein design

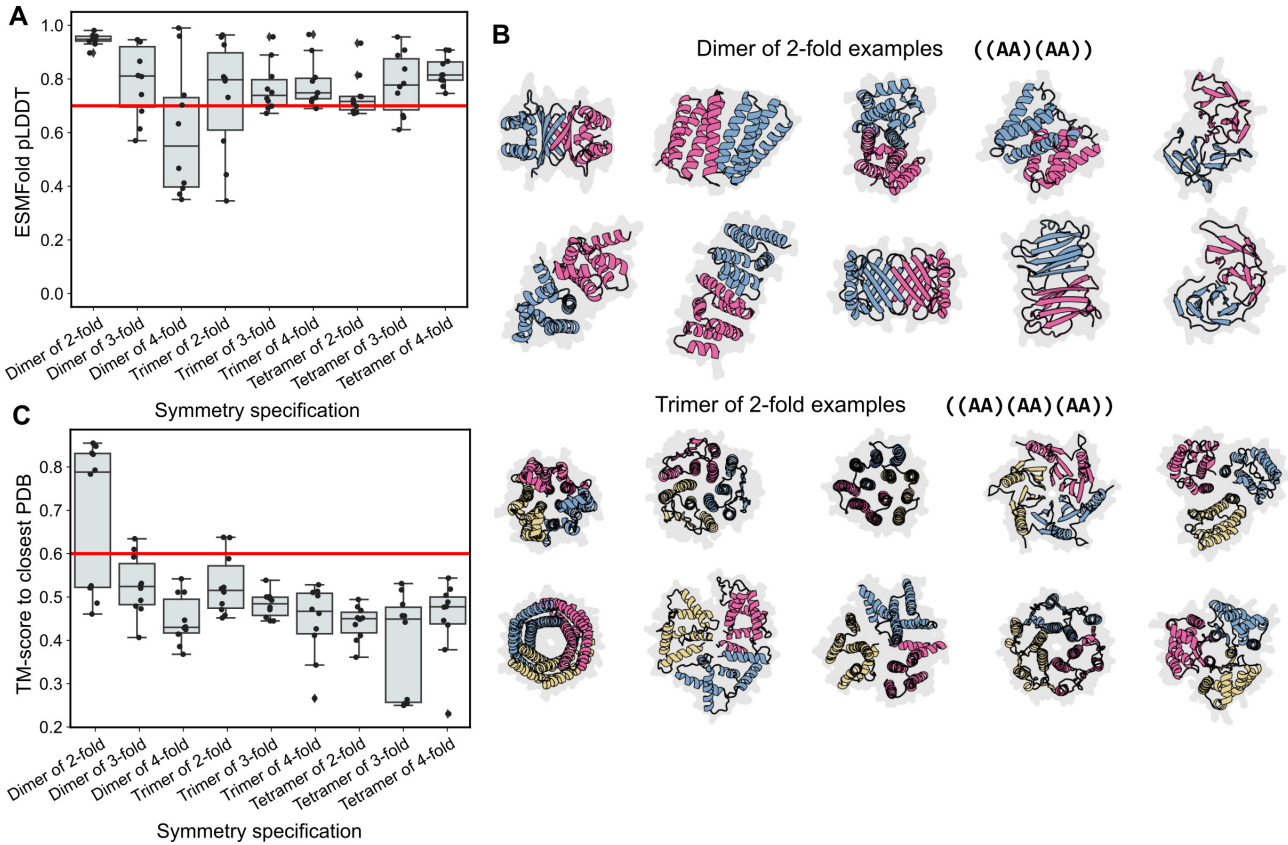


Figure S3. Additional plots for the design of two-level symmetry. (A) ESMFold pLDDT values for different two-level symmetry design specifications (10 seeds per specification). A red line is plotted at pLDDT = 0.7. (B) All ten of the designs of a dimer of 2-fold and of a trimer of 2-fold symmetry. (C) TM-scores for different two-level symmetry design specifications (10 seeds per specification); the TM-score is between the best design and the closest structure in the PDB. A red line is plotted at TM-score = 0.6.

用于生成蛋白质设计的高级编程语言

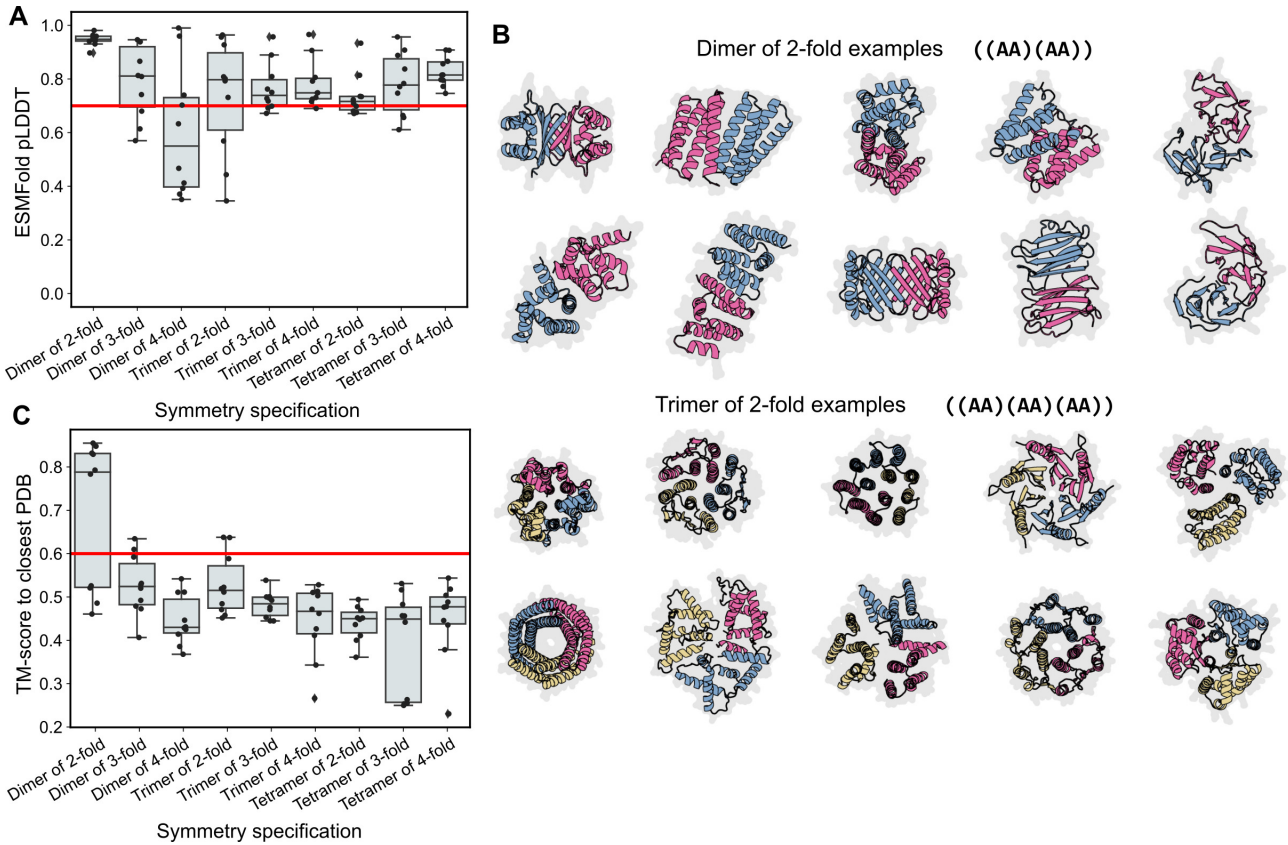


图 S3. 双层对称设计的附加图。(A) 不同双层对称设计规范的 ESMFold pLDDT 值（每个规范 10 个种子）。一条红线在 pLDDT = 0.7 处绘制。(B) 2倍对称的二聚体和 2倍对称的三聚体的十种设计。(C) 不同双层对称设计规范的 TM分数（每个规范 10 个种子）；TM分数是在最佳设计与 PDB 中最接近结构之间的值。一条红线在 TM分数 = 0.6 处绘制。

A high-level programming language for generative protein design

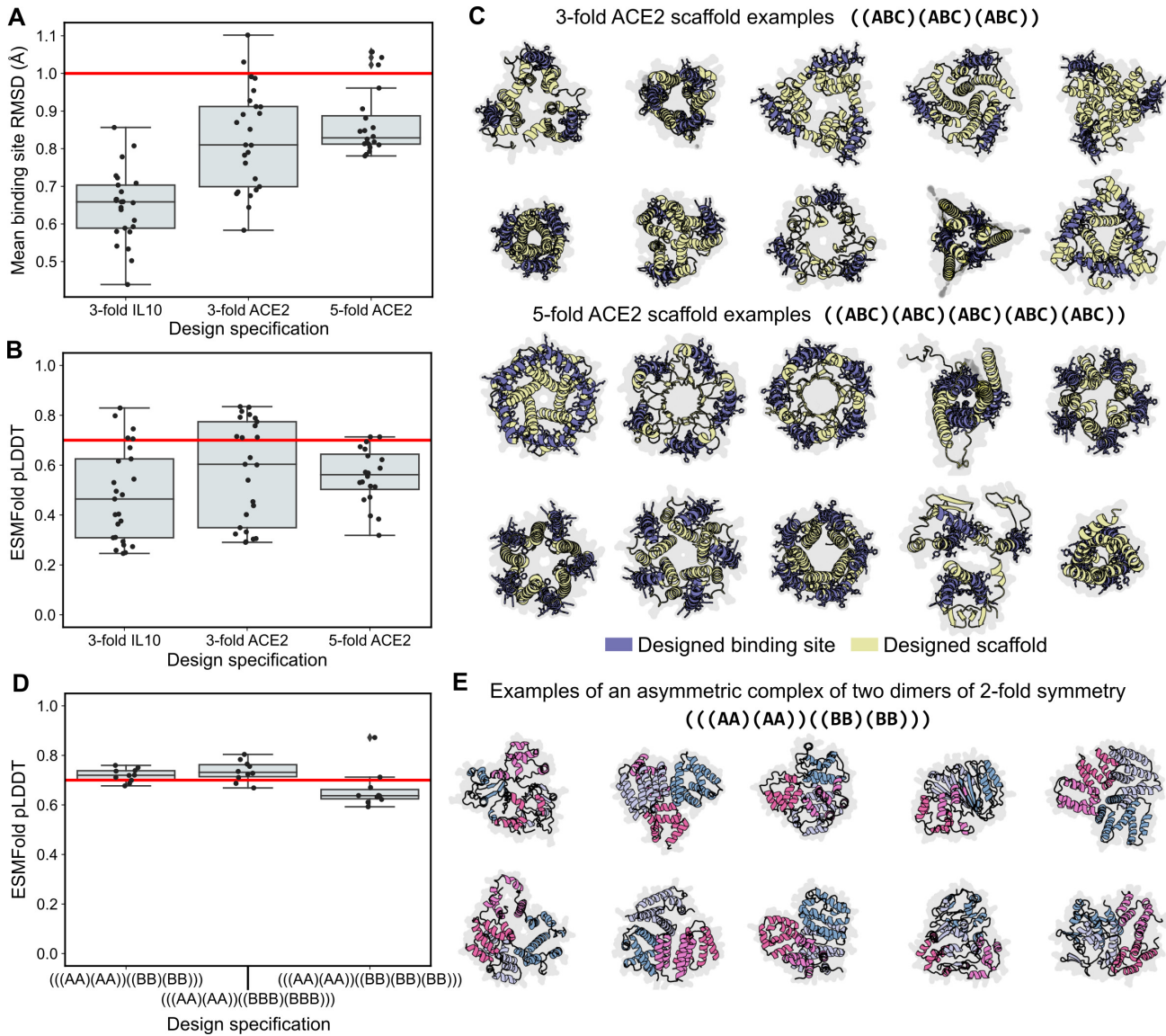


Figure S4. Additional plots for multi-level hierarchical design (A) ESMFold pLDDT values for different binding site scaffolds with fold symmetry (20 seeds per specification). A red line is plotted at pLDDT = 0.7. (B) RMSD values for different binding site scaffolds with fold symmetry (20 seeds per specification). The mean RMSD is reported across either three or five binding sites. A red line is plotted at pLDDT = 0.7. (C) Ten randomly sampled designs for the design of 3- and 5-fold symmetric ACE2 binding site scaffolds. (D) ESMFold pLDDT values for different asymmetric-symmetric design specifications (10 seeds per specification). A red line is plotted at pLDDT = 0.7. (E) All ten of the designs of an asymmetric complex of two dimers of 2-fold symmetry.

用于生成蛋白质设计的高级编程语言

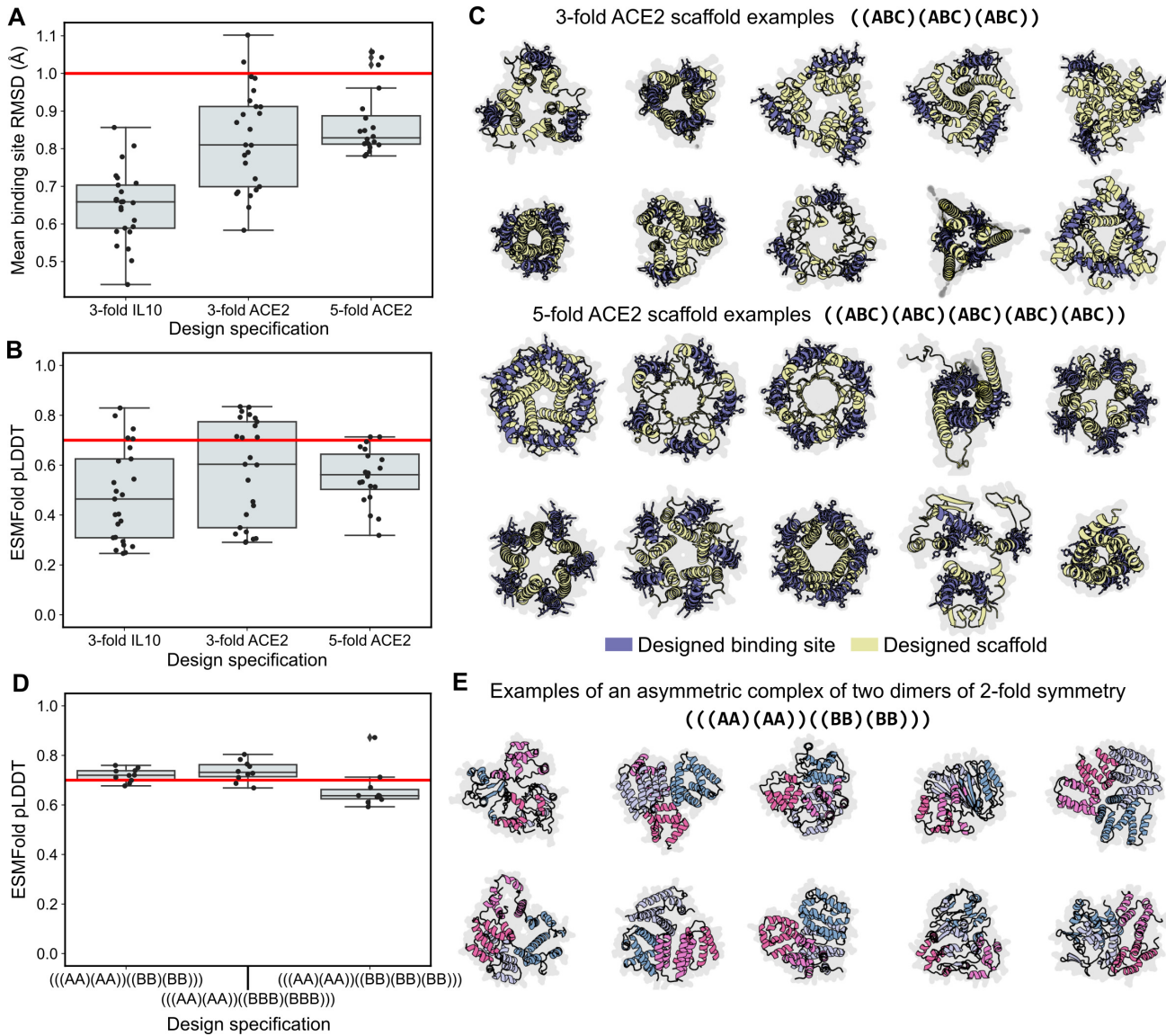


图 S4. 多层次层次设计的附加图示 (A) ESMFold pLDDT 值对于具有折叠对称性的不同结合位点支架（每个规格 20 个种子）。在 pLDDT = 0.7 处绘制了一条红线。(B) 对于具有折叠对称性的不同结合位点支架的 RMSD 值（每个规格 20 个种子）。报告在三或五个结合位点之间的均值 RMSD。在 pLDDT = 0.7 处绘制了一条红线。(C) 随机抽取的十个设计用于设计 3- 和 5-折叠对称的 ACE2 结合位点支架。(D) ESMFold pLDDT 值对于不同的非对称-对称设计规格（每个规格 10 个种子）。在 pLDDT = 0.7 处绘制了一条红线。(E) 两个具有 -折叠对称性二聚体的非对称复合体的十个设计。